

Distributed Systems - Big Picture

Contents

What is a Distributed System.....	3
Distributed System	3
Types of distributed systems	3
Demo	4
Adopting the Traditional Model.....	6
DNS	7
CDN	7
Vertical Scaling.....	8
Horizontal Scalability.....	10
Auto scaling	10
Demo - first design.....	12
Components of a Distributed System	13
Front-end Layer	13
State Management	13
Major components of the front-end	20
Backend - Handling Business Logic with Web Services	23
Data Layer.....	30

Replication	31
Sharding / Partitioning.....	34
CAP Theorem	36
Extra materials.....	37
Communication	38
Synchronous communication	38
Asynchronous communication	39
Message queues	40
Distributed Systems in the Real World	46
Scaling Teams and Organization	47
Automating Testing and Deployment	48
Setting up Monitoring and Alerting	49
Collecting and Viewing Metrics.....	51
Frameworks and libraries for developing distributed systems	53
Links	54

What is a Distributed System

Distributed System

https://en.wikipedia.org/wiki/Distributed_computing

Collection of independent computers that appear to its users as one computer.

Characteristics of a distributed system:



- All servers are processing requests concurrently.
- Failure of one server should not affect other servers.
- Servers maintain their own time using local clocks. Global clocks is hard to implement (to synchronize all the servers).
Workaround: [Logical clocks](#) - implementing a protocol on all machines within your distributed system, so that the machines are able to maintain consistent ordering of events.

Types of distributed systems

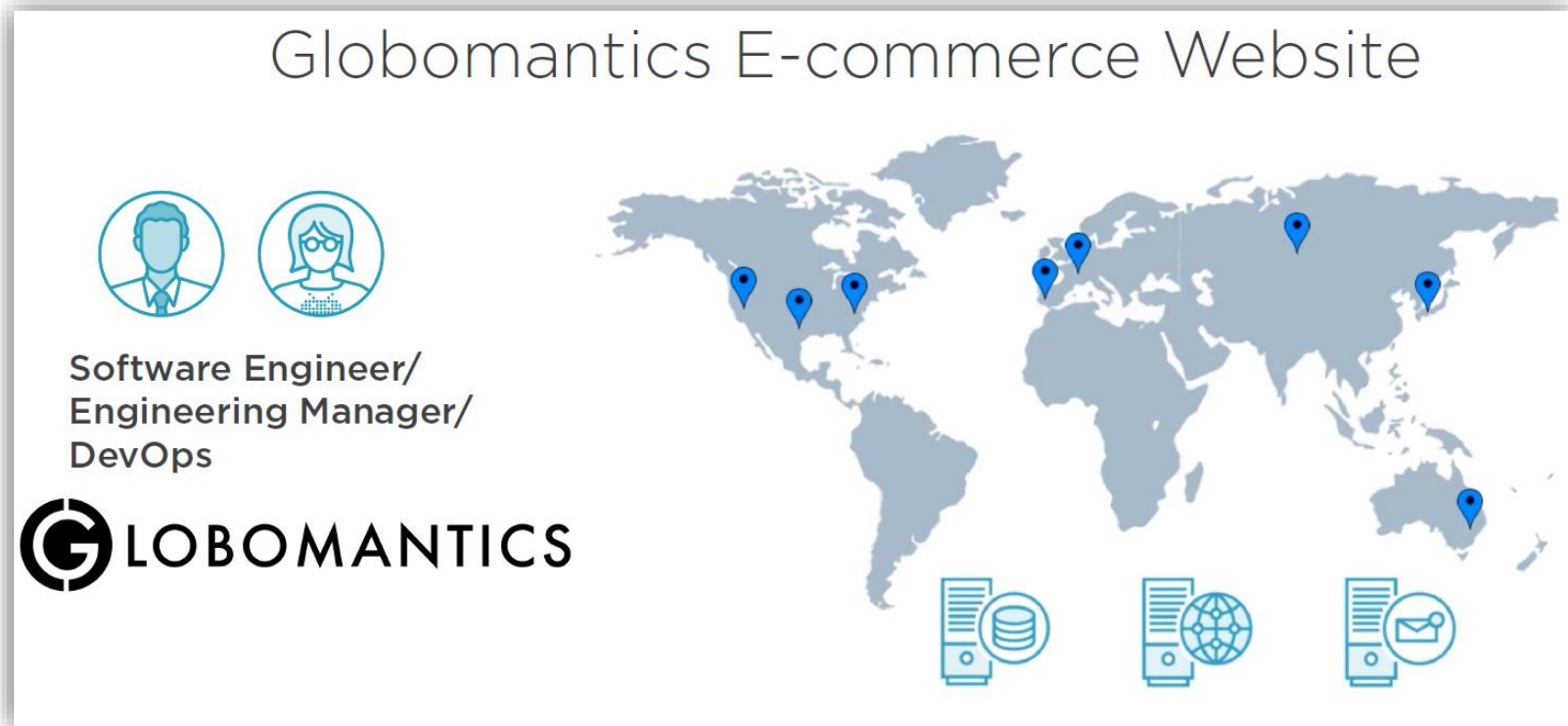
Client-server: The client requests the server for resources or a task to do, the server allocates the resource or performs the task and sends the result in the form of a response to the request of the client.

Peer-to-peer: Each device can operate as both the client and server. Computer network applications use a peer-to-peer system to organize processors that communicate with each other but maintain separate local memory bases. The programs and servers in this network all have the same privileges, access and functions and communicate at the same level without a hierarchy.

Three-tier: client-server with separated presentation layer, application layer and data layer.

N-tier: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier> - divides an application into logical layers and physical tiers. Tiers are physically separated, running on separate machines. A tier can call to another tier directly, or use asynchronous messaging (message queue).

Demo



Your objective is to research on how to build a platform that facilitates the sale of items between buyers and sellers by connecting them on your website.

Moreover, once you've successfully launched your website in the US, you expect it to grow across multiple countries in the future. Put yourself in the shoes of a software engineer, an engineering manager, or an architect.

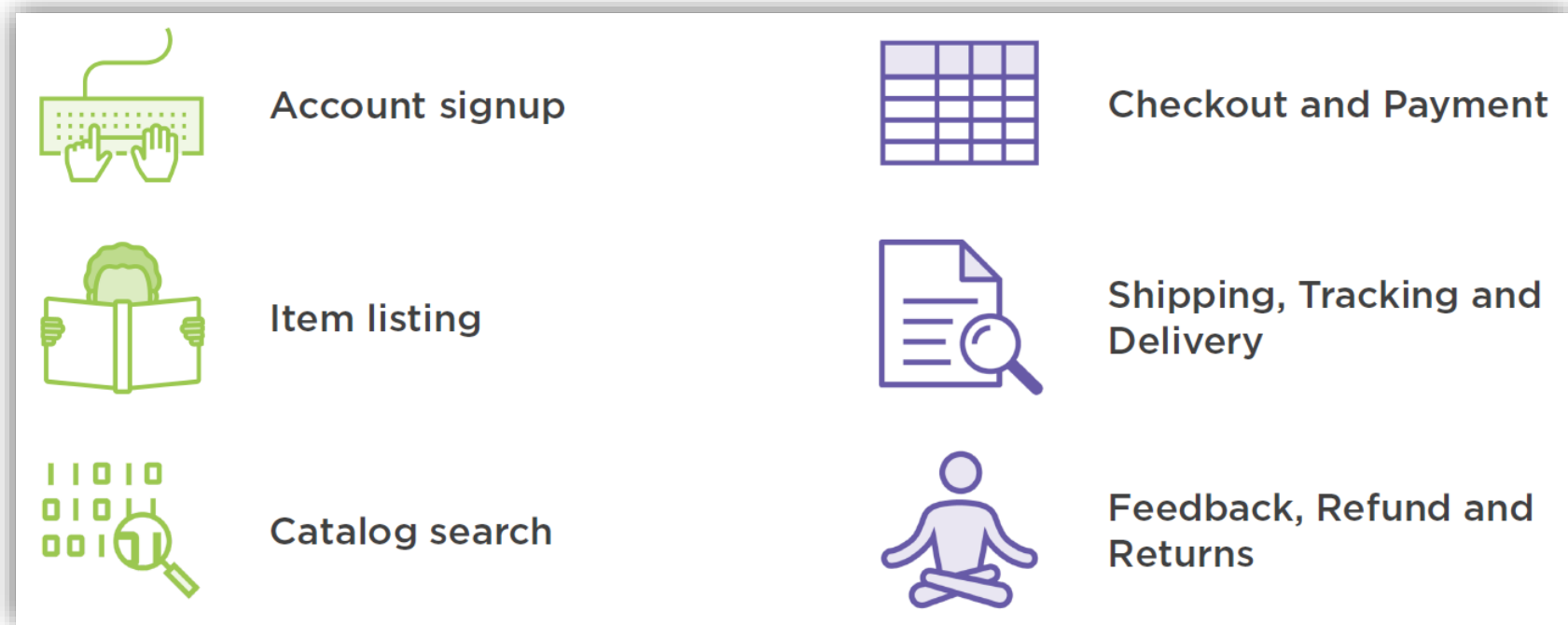
How would you go about architecting such a system? This brings us to an important design principle. **Application architecture should evolve around the business model** and not around the technology.

Whenever you set out to design a system, you shouldn't really be focusing on what framework or technology you'd be using, but the primary domains and business models essential to your application.

This is also known as [domain-driven design](#). For our Globomantics E-commerce website, you would list the primary use cases it should support:

Primary use cases:

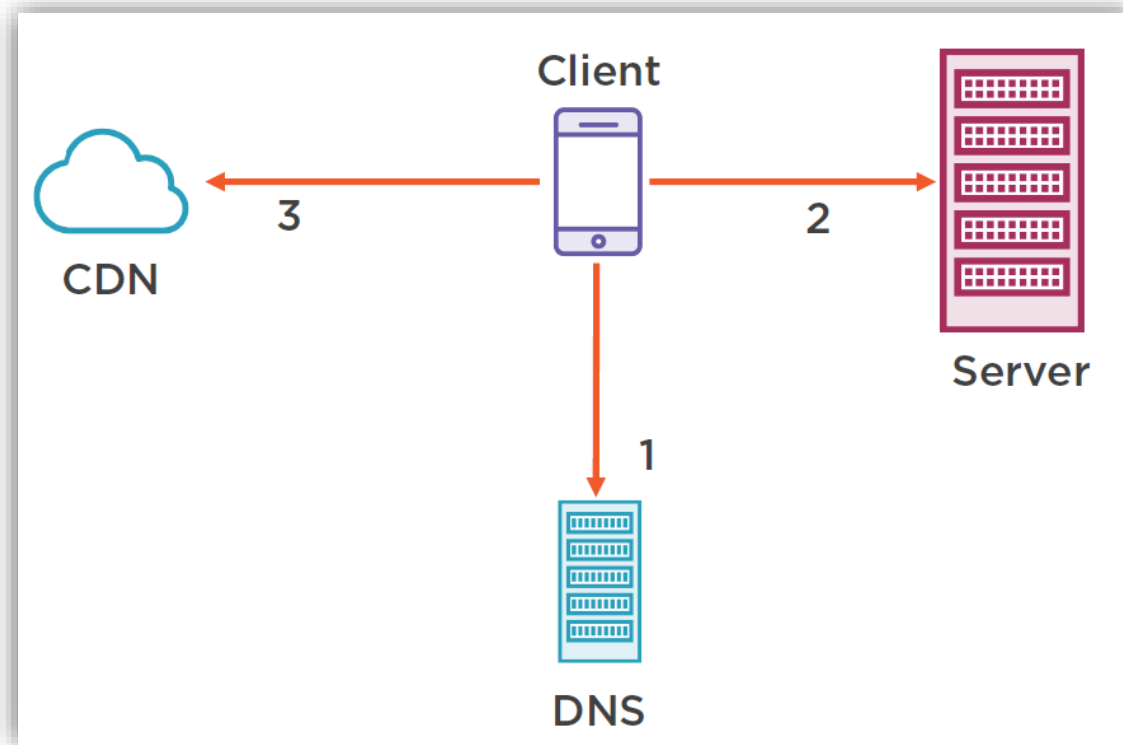
- You'd like the users, buyers, and sellers to be able to sign up for an account.
- A seller should be able to list the items that they would like to sell. A buyer should be able to search the catalog for items that they may be interested in.
- As soon as they find something, they should be able to add items to their cart and complete checkout and payment. After their order is completed, they should be able to track their order delivery.
- Next, buyers and sellers should be able to leave feedback for each other. Finally, if needed, buyers should be able to return an item and get a refund from the seller.



Adopting the Traditional Model

If you adopt the traditional client-server model, your entire application with the database will run on a single server. The client could be a browser:

- It first connects to the DNS server.
- Next, the client may request for the home page that in turn will require loading of additional resources like images, videos, CSS files from your CDN provider.



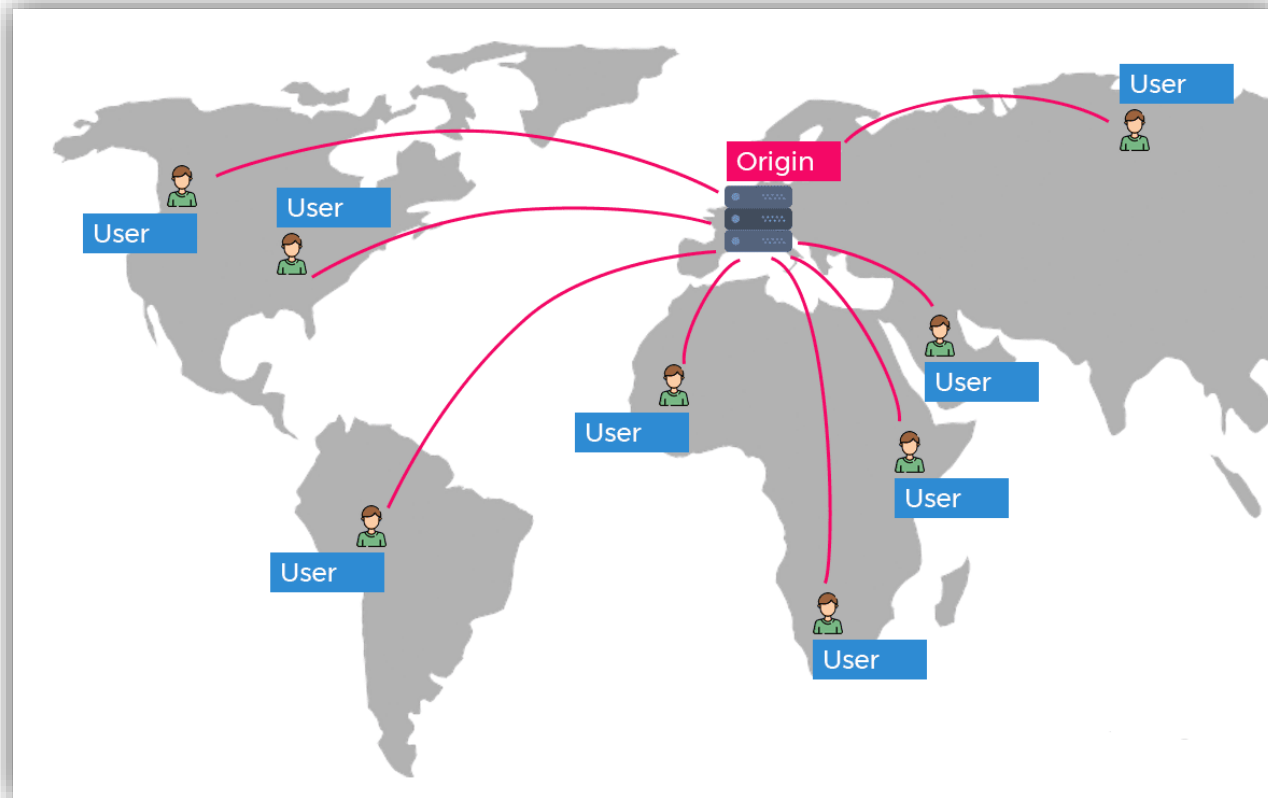
DNS

Domain name system. It's a hierarchical and decentralized naming system that translates a domain name to an IP address.

CDN

[Content delivery network](#), is a collection of distributed servers that deliver static pages and other content to a user based on the geographic location.

This greatly reduces the amount of bandwidth and the page load times as the resource is located closer to the client.



Vertical Scaling

If there's a sudden increase in the number of requests, you can add more resources by replacing your current server with a more powerful one. You can add more RAM, more hard drive for storage, and so on.

This is known as vertical scalability. Vertical scalability allows you to scale by simply upgrading the server hardware resources and our network throughput.

Advantages:

- The biggest advantage of vertical scaling is that you do not need to rearchitect your application to make use of these additional resources.
- Additionally, if your application is hosted on a virtual private server instead of dedicated hardware, vertical scaling can be performed in a matter of minutes.

Vertical Scalability	Upgrade hardware and/or network throughput No application redesign needed Works even better with Virtual servers
-------------------------	---

Disadvantages:

- Adopting vertical scaling does come with its own cost. Firstly, after a while, the cost associated with a hardware upgrade might be significantly more than the benefits they provide.
- Moreover, there are strict limits on how many hard drives or how much RAM you can add to your current system.
- No matter how much money you are willing to spend, after a certain point, there is no hardware available that can help you scale vertically.
- The operating system or your application may be designed in a way that doesn't support vertical scaling. So even if you add more CPUs, the operating system may only support a certain number of execution threads at a time.

In spite of these limitations, vertical scalability could be a great option if your application is relatively simple in scope and you can afford the cost of ensuing upgrades.

Horizontal Scalability

To scale horizontally, you run each of your application components on multiple servers.

Auto scaling

If there is a sudden spike in traffic, you can add more servers, and when there is a drop, you remove servers. And once you have set up the proper configuration, all of this can happen automatically.

This is also known as [auto scaling](#). Another thing to note is that you achieve all of this with cheap commodity hardware.

Advantages:

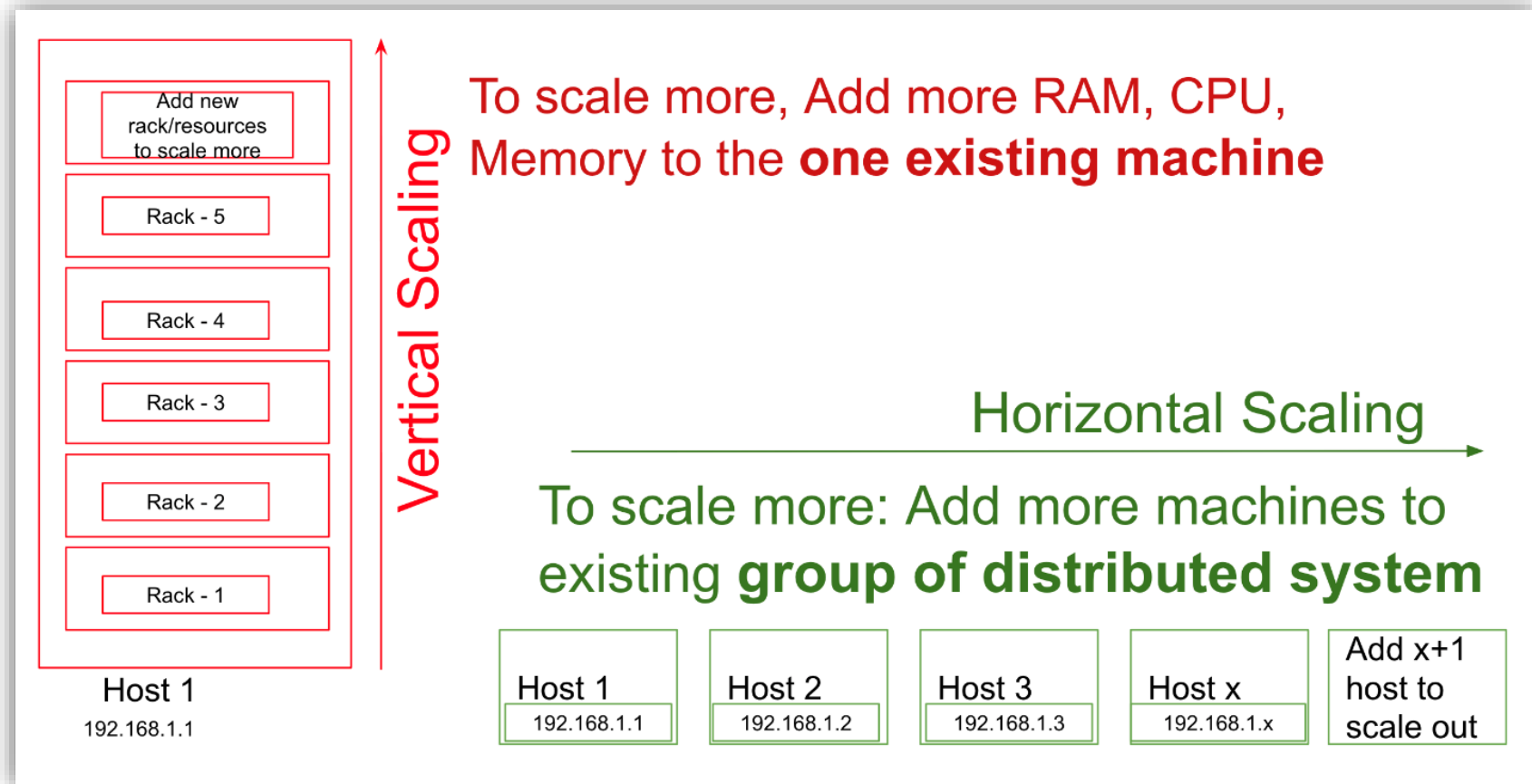
- flexibility
- cost effectiveness

Disadvantages:

Horizontal scalability has its own set of challenges. Specifically, it can add an incredible amount of complexity to your application.

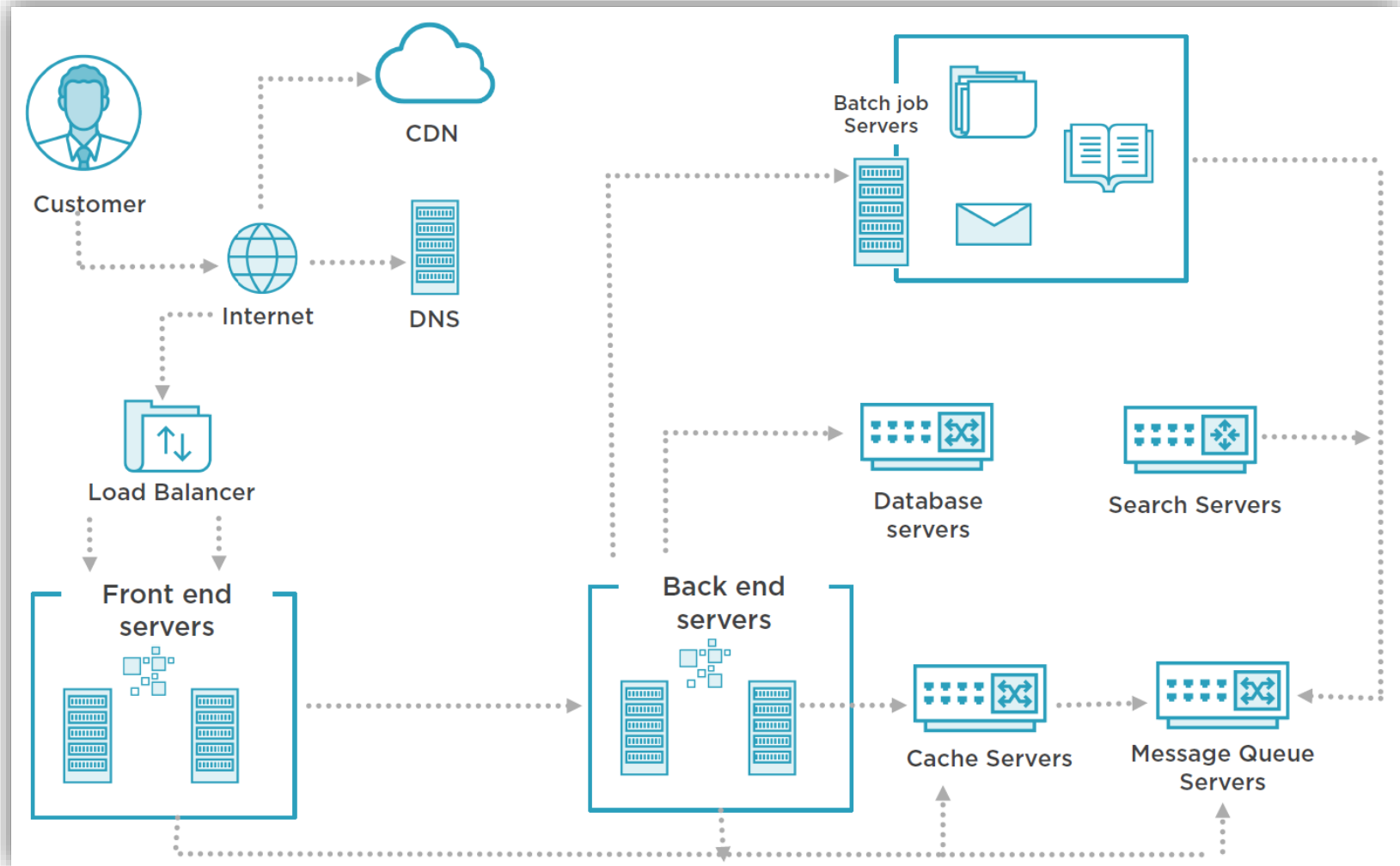
Horizontal Scalability	Run each component of your application on multiple servers Add/Remove servers as needed Commodity hardware instead of specialized servers
---------------------------	--

Horizontal vs Vertical scaling



We can also scale databases - see chapter Data Layer.

Demo - first design



Let's look at what Globomantics distributed system might look like. This is a high level view of a distributed system. It shows the different components that are part of most distributed applications and how the data flows between them.

Components of a Distributed System

Not all Distributed Systems look the same. If your application is primarily a search engine, its design will be very different from a video-sharing platform.

Main requirements are similar:

- scalability
- high availability
- resilience to failures

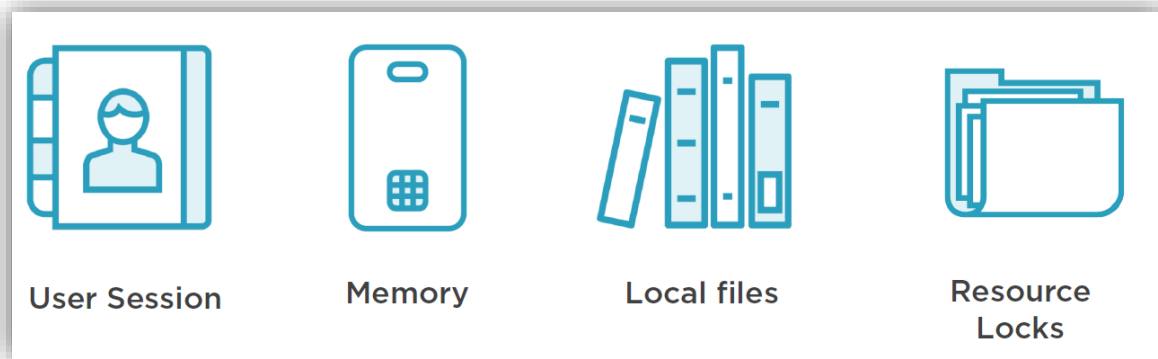
Following are some of the components of most Distributed Systems.

Front-end Layer

The frontend is the entry point of the application. As the majority of your request and responses have to pass through the front-end layer, it receives a lot of traffic and needs to withstand the highest level of concurrency. The frontend intercepts most of the user requests, it then delegates these requests to the back end to execute any business logic.

State Management

In order to scale your frontend, you have to know how to manage your state. But what qualifies as state? State could be information stored in the user session, the memory, local files, or resource locks.



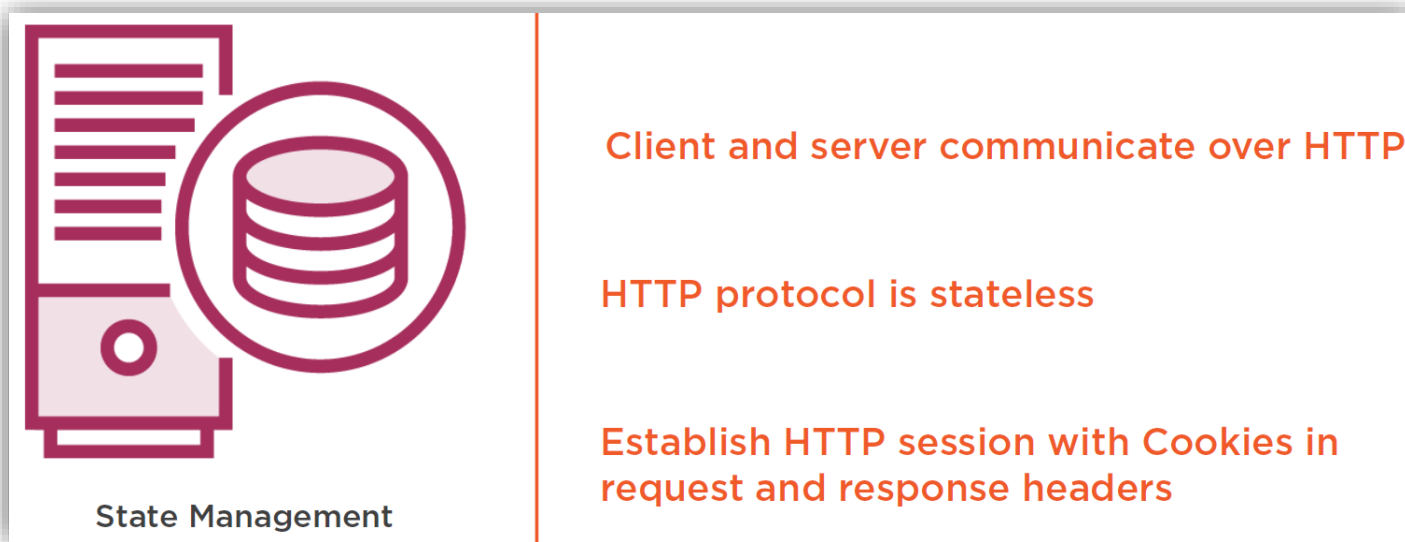
Horizontal scalability

By moving all of the state data out of the front-end servers, you can scale the frontend by adding more servers. This is also what helps us to set up auto scaling. So anytime you're thinking of designing the frontend, always think of how you can push the state outside of the server. In this way, scaling is possible by simply adding more servers. **Statelessness is what allows these servers to be completely interchangeable.** The client doesn't care which server processes its request. As a consequence, your servers are decoupled from the client.



Keep your Servers Stateless.
Scale Horizontally by
adding more Servers.

When a browser sends a request to the Globomantics front-end, the state is transmitted over HTTP. HTTP stands for Hypertext Transfer Protocol. It's important to note that **HTTP is stateless**. If the browser sends another request to the same server, it is treated as an entirely new request. It is the application's responsibility to find a way to persist the state between subsequent requests.



Sessions

One way to achieve this is to set up an HTTP session with cookies. The browser sends an HTTP GET request to the server for the very first time. At this point, the HTTP session headers Set-cookie field will be EMPTY. The server then generates a unique ID and assigns it to the cookie field. This field is then sent back to the client as part of the response header along with any other session data. All subsequent requests will contain this unique ID as part of the request header.



State Management

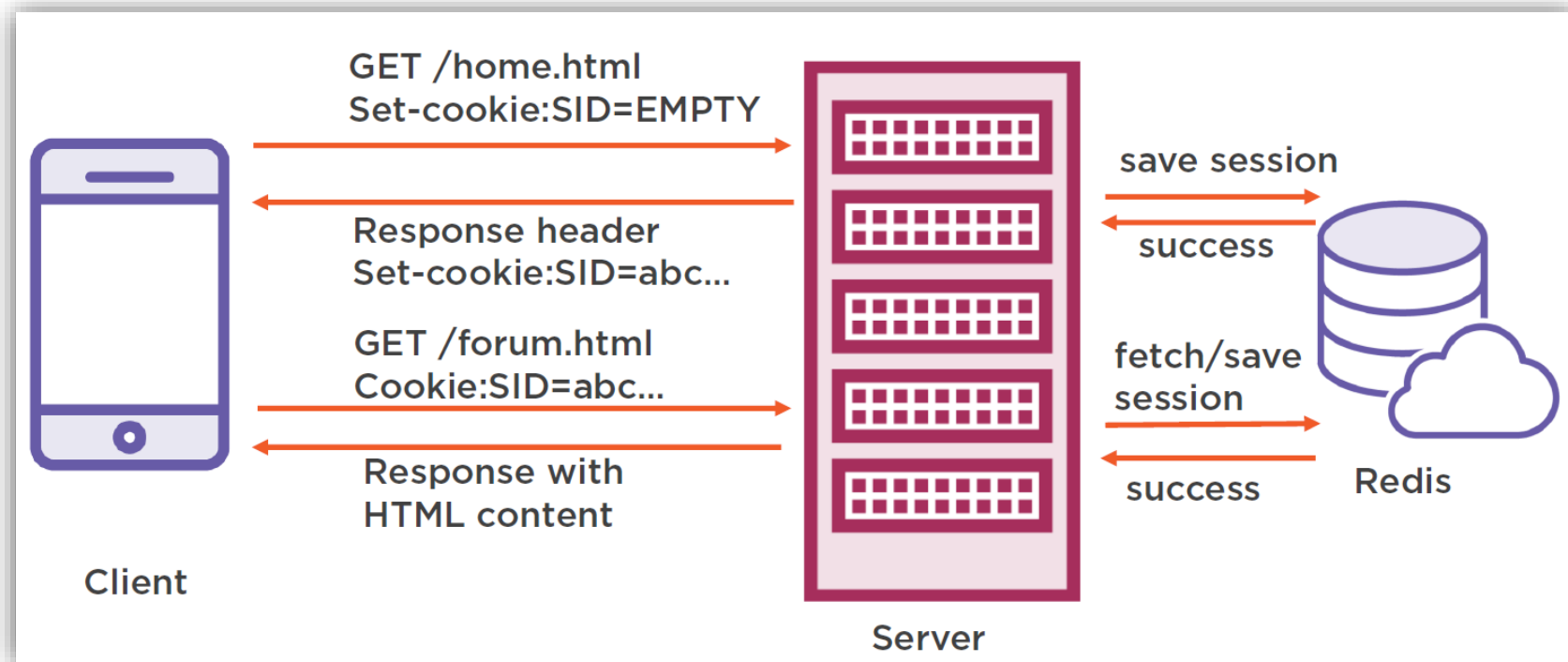
Cookies sent with every individual request

Store session data in an external datastore

For local files, use CDN or an Object store

A disadvantage of this approach is that cookies are sent with every single request. Even if it's for an image, style sheet, or a font file. You can see how this can quickly become an expensive operation.

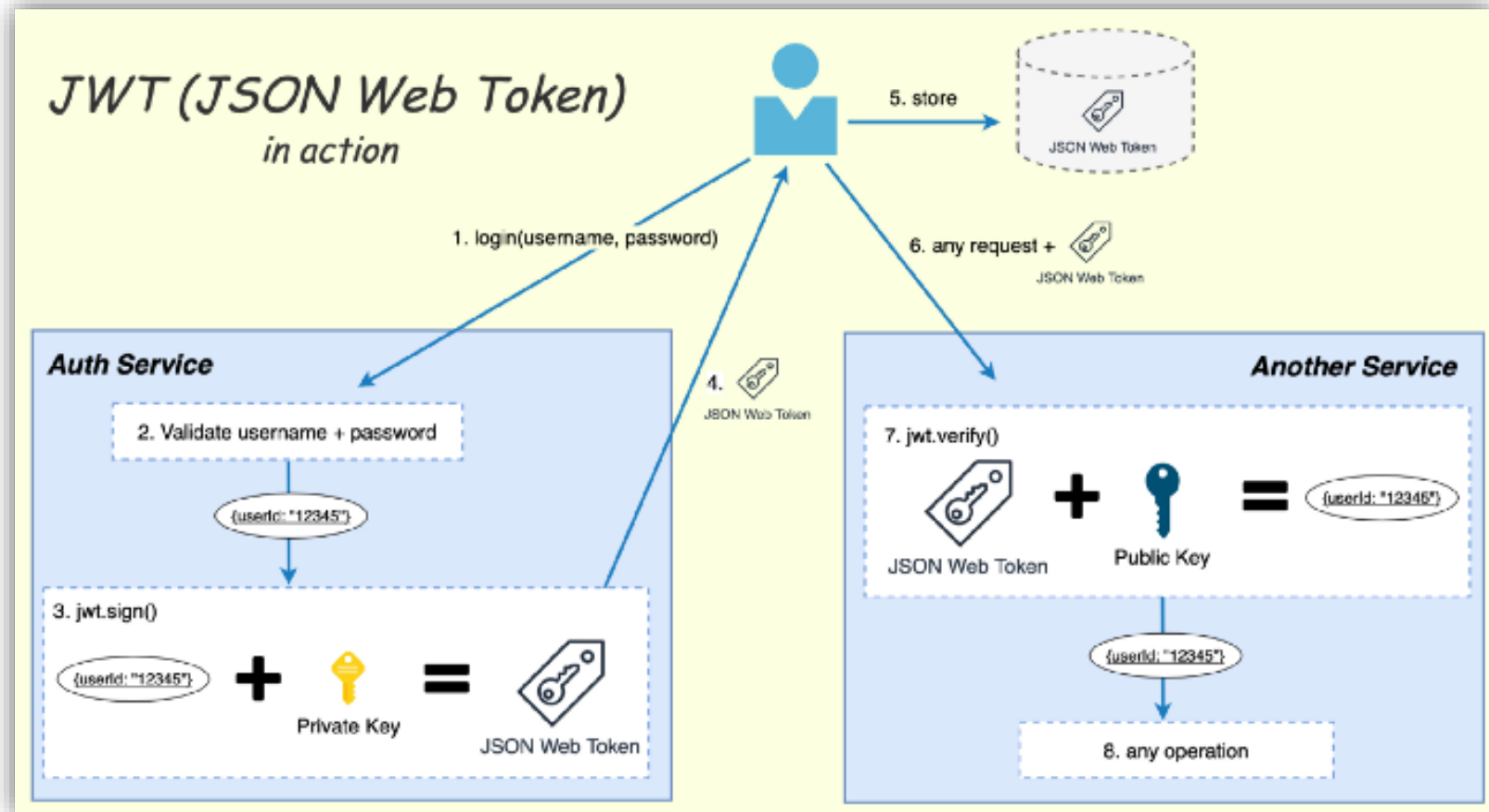
To get around this, we can use an external datastore like Redis (<https://redis.com/solutions/use-cases/session-management>) or Memcached.



For static, public files you can use a CDN provider like Amazon CloudFront. You could also use a distributed file storage like Amazon Simple Storage Service for public and private files. They are inexpensive and ideal during initial stages of development. Additionally, they also have great support for implementing distributed locks.

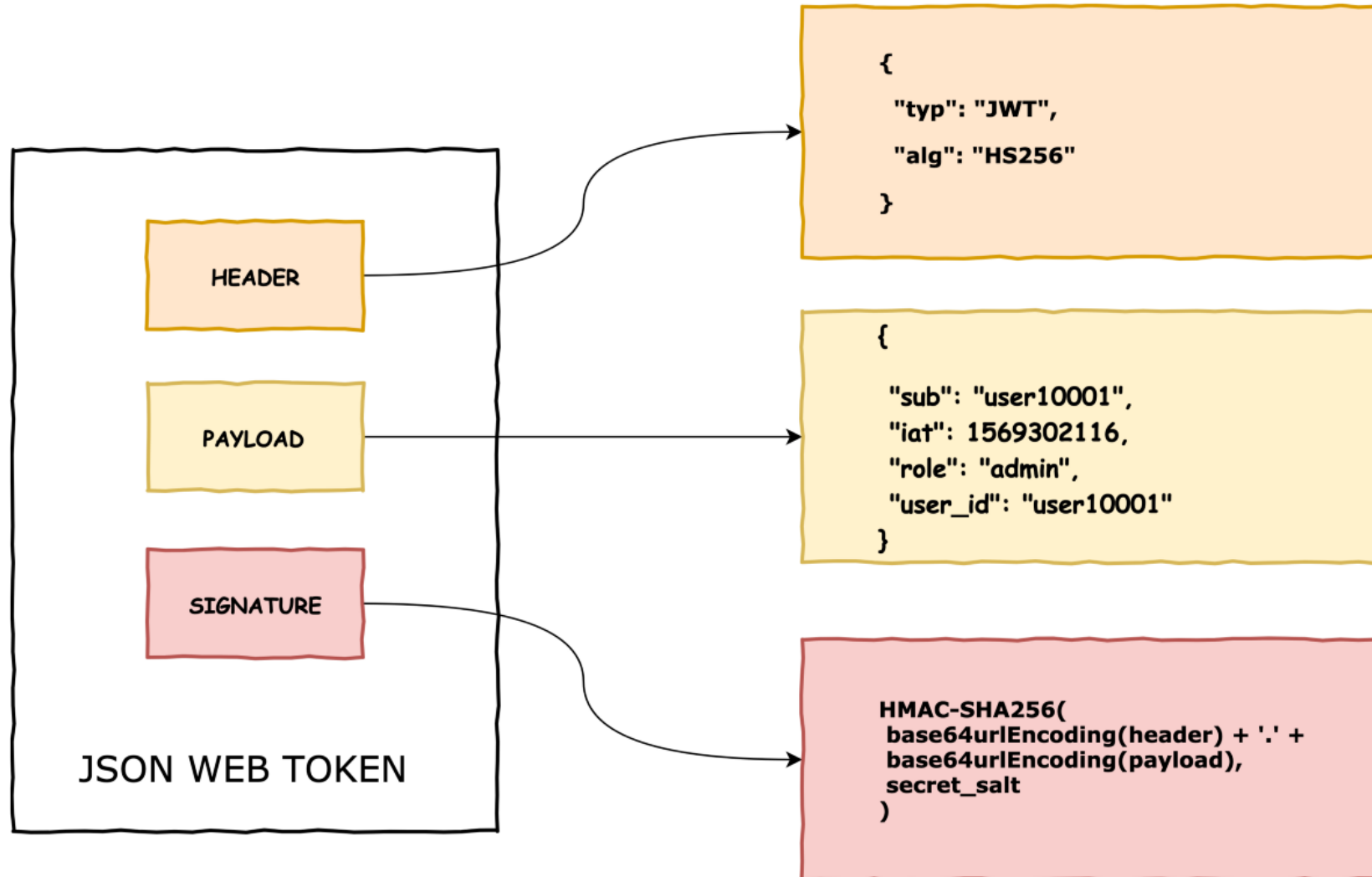
All these approaches can cause an increase in latency as your server now needs to connect to an external data store for something that was available locally. If you design your application knowing these limitations, your frontend can scale with relative ease.

JWT - Json Web Token

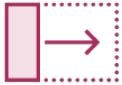


Using JWT is stateless on the server side - there is no need to store it on the server. The client needs to use JWT in the requests.

JWT contains a payload that is encoded and can be used to share info between the server and client. The servers only need to store the secret for the JWT signature.



Major components of the front-end

	DNS - Domain Name System (eg. Route 53, easydns.com)
	CDN - Content Delivery Network (eg. Akamai, Amazon CloudFront)
	Load Balancer (eg. Elastic Load Balancer, HAProxy, Nginx)
	Caching (eg. Browser, Redis/Memcached)

DNS: It takes a domain name like `www.globomantics.com` and returns an IP address like `152.211.342.199`.

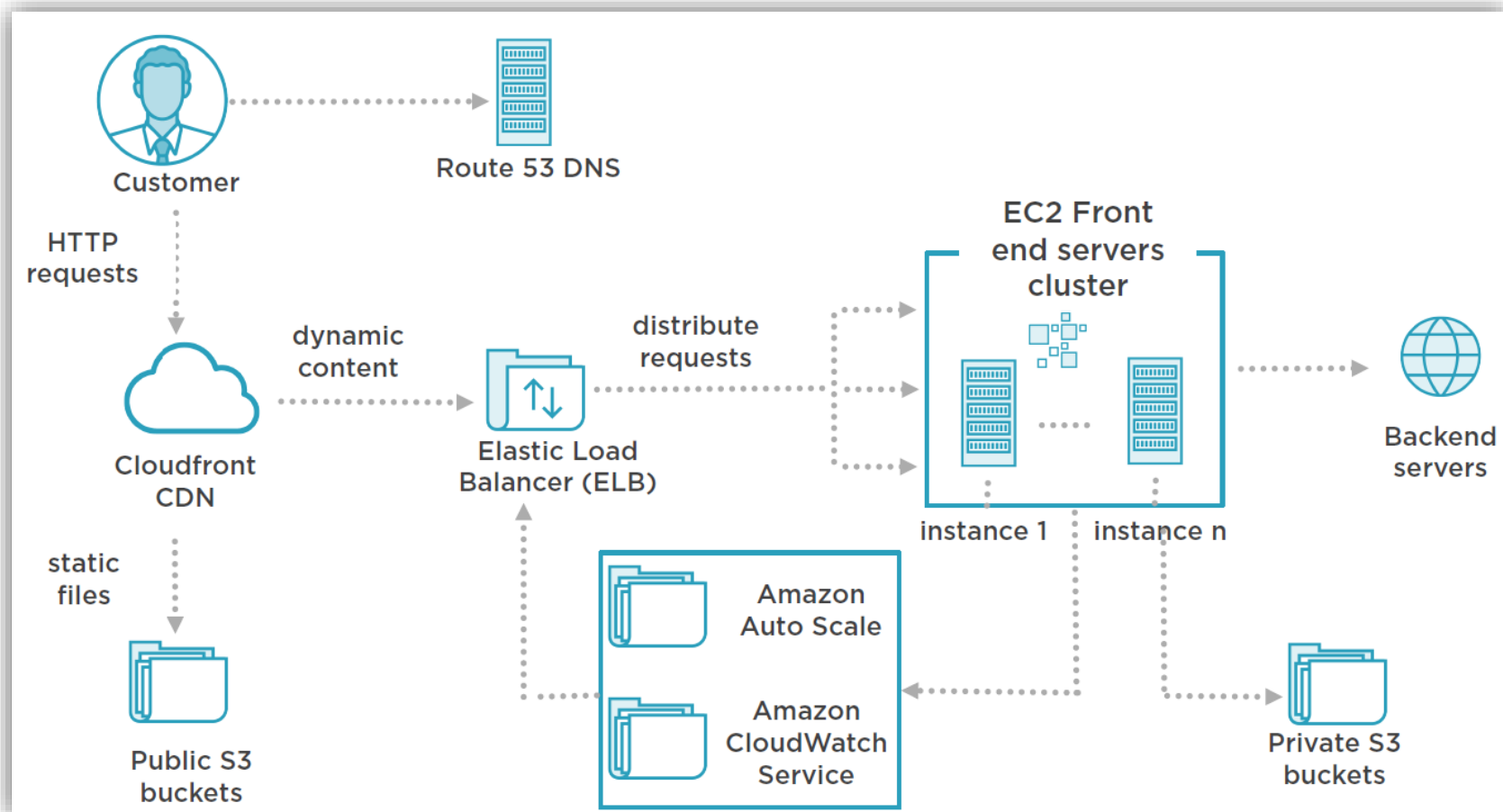
CDN: improving latency for serving static pages based on your geographic location. CDN also allows caching of entire pages, reducing the overall load on your front-end service; however, if your application is primarily serving dynamic content, then CDN may not be the right choice for caching.

Load balancers

can be set up as the entry point for your data center separating your clients from the front-end web service. They not only distribute the incoming traffic across the web servers but also prevent exposing the web servers to all the clients. This is especially useful for detecting Denial-of-Service attacks. Additionally, they can also perform SSL termination allowing connection from the load balancer to the web server to be HTTP instead of HTTPS. This greatly reduces the overhead on your web service. Load balancers come in both hardware and software form. Hardware load balancers can be very expensive. If you're looking for a software solution, then Nginx is a very popular choice. Amazon AWS comes with its own load balancing solution called the Elastic Load Balancer.

Cache

is one of the most important components of the front-end layer. Sometimes scaling can be achieved simply by caching responses instead of adding new servers. We've already looked at how CDN can help with caching static content. For dynamic content, you can deploy reverse proxies like Nginx and Varnish. For single-page applications, using the browser localStorage can greatly minimize the number of app server requests and provide a great user experience. Let's look at how we can take everything that we have learned to deploy our frontend in AWS:



- DNS routing and load balancing has been taken care of by AWS.
- Your responsibility is to create, run, and manage the EC2 instances shown here.
- Additionally, you need to configure the various settings around auto scaling, load balancing, where data will be stored, and so on.

When a user visits the Globomantics website, they will first be redirected to the Route 53 DNS. Later, your web request also passes through the Elastic Load Balancer where SSL termination or auto scaling can be implemented. These components will handle all your scalability and high availability needs. There is no need to worry about manually scaling or providing any redundancy. Finally, your requests are processed by the front-end EC2 server instances. They talk to the back-end servers to render the response. Again, we move any data related to the state out of the front-end servers to the private and public S3 buckets. Part of your response may be cached by the CloudFront CDN depending on your geographic location.

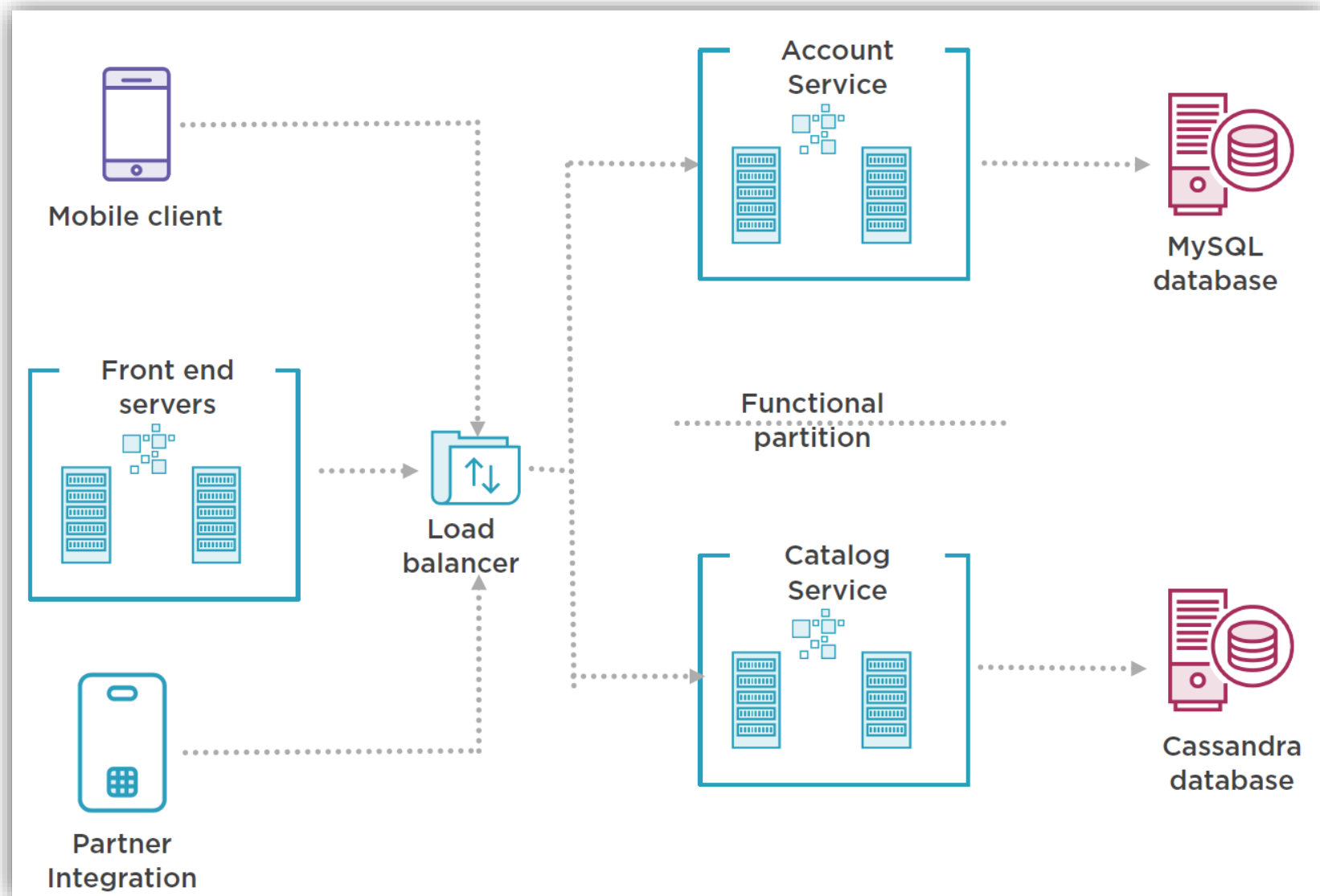
Backend - Handling Business Logic with Web Services

The part of the application that encodes the business rules and determines how data is created, stored, or modified is known as the business logic. This is different than the part of the application that stores the data or renders the user interface. The way you implement the business logic is with web services.



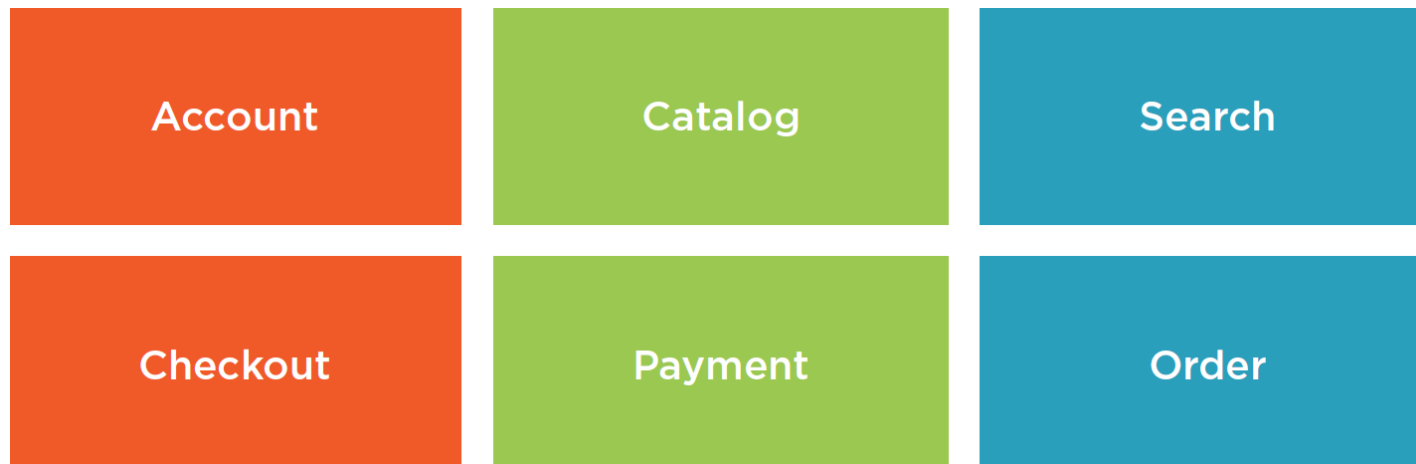
Web Services encapsulates
the business logic and hides
complexity behind an API
contract

The frontend is one of the clients for your business layer. You can have other clients like a partner service or a mobile device. Also, it's very common to have another load balancer between your frontend and back-end layer.



First, let's look at web services. Web services encapsulates the business logic and hides the complexity behind an API contract. In other words, your clients are decoupled from the business layer. You define your API first and then implement the client and web services at the same time. As long as the API contract doesn't change, you can add and remove clients or change the web service implementation without affecting the client. Let's see how we can apply this to our Globomantics E-Commerce application.

We have a small subset of the services that our platform should support. These services are modeled after our business domain. At the very least, we'll need an account service where users can sign up and log in, a catalog service containing the inventory of all items on our website, a search service to discover these items, a checkout service that allows a buyer to add items to their cart and begin the checkout process, a payment service that allows the buyer to pay the seller for the items they bought, and finally, an order service that provides details associated with an order.



Functional partitioning

The process of splitting a large system into a set of smaller, loosely coupled, independent web services is known as functional partitioning. Each service focuses on a subset of the overall system functionality.

In our diagram, we have three clients, a mobile device, the front-end layer, and a partner service. Each of them make requests that are first intercepted by one or more load balancers. This request is then distributed among the back-end servers. Each service has its

own set of servers partitioned by the functionality that they provide. These servers also have different scalability needs. The account service, for instance, will have both read and write requests. The catalog service, on the other hand, will primarily have read requests. This allows you to scale them independently. These services can also have data suitable for different data models. The account service can store its data in a relational MySQL database. The catalog service can store its data in a NoSQL Cassandra database.

Implementing web services

We'll be using [REST](#) to create these services. REST is a resource-oriented architectural style that defines a set of constraints used for creating web services. If we wanted to model resources for Globomantics using REST, we can expose a REST API to allow clients to search for orders given an order ID. We make an HTTP GET request to get details for order OrderID 123456. The URL is how we uniquely identify a resource like order.

<http://globomantics-web-site.com/order/123456>

The server returns a JSON response with the order details like OrderID, Date, and Status.

JSON Response

```
{  "OrderID": "123456",  
    "Date": "09/12/2020",  
    "Status": "Delivered"  
}
```

In the same way, you can expose additional resources like account, item, and shopping list. We won't go into too much detail on REST here. But the reason why REST is so popular is that it can take advantage of many of the underlying features that come with the HTTP protocol, things like PUT, POST, GET methods to modify or retrieve resources. The range of different HTTP status codes to communicate if a request was successful, redirected, or failed due to a client or server error. You can use the HTTP security controls to secure your communication. You can use HTTP caching proxies like Varnish or load balancers like mod_proxy. This is especially useful when you need to scale your web services. You can even use the large ecosystem of monitoring tools that support HTTP out of the box. All of these make REST over HTTP an excellent choice for implementing your web services.

Why REST?

Leverage HTTP features (PUT, GET, POST)

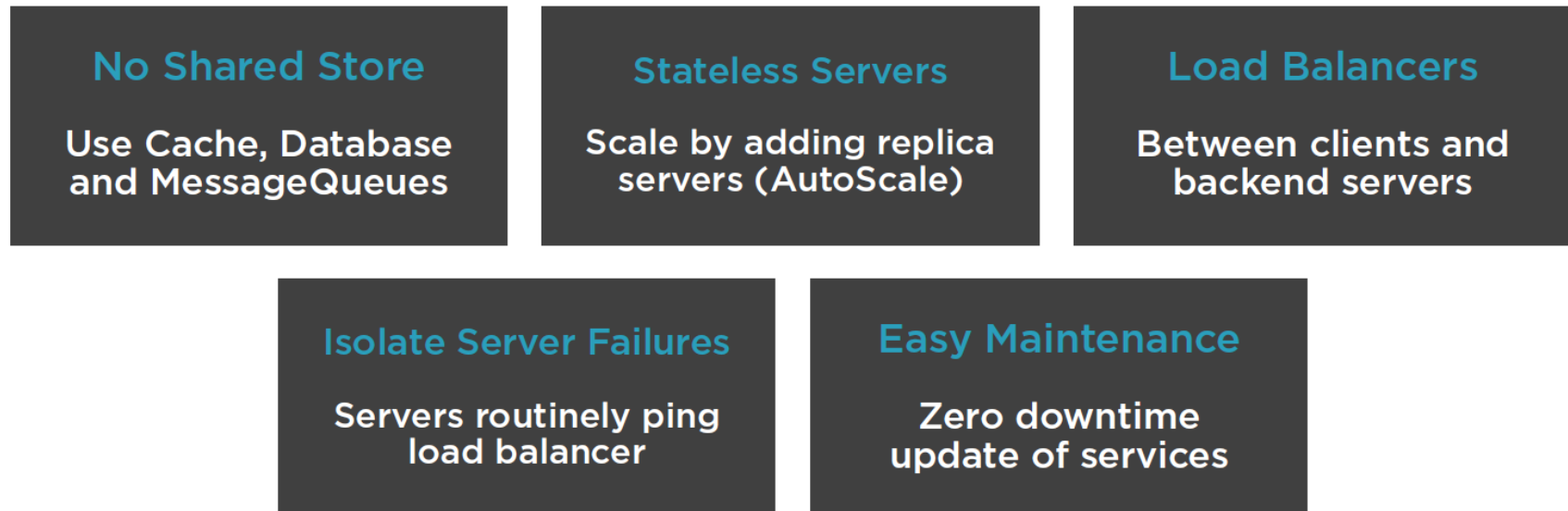
Status codes (200 - OK, 400 - Bad Request)

Security Controls

Caching

Ecosystem of Monitoring tools

Horizontal scaling



Scaling the business layer is similar to scaling the frontend in a lot of ways. Identify the data representing the state and move it out to a shared storage. This could be a cache, a database, or even a message queue. Once this is done, your back-end servers are essentially stateless, allowing you to take advantage of the powerful features provided by your hosting provider, like autoscaling. You can set up load balancers to distribute the incoming traffic among your back-end servers. Moreover, each of these servers regularly ping your load balancer, allowing you to perform heartbeat checks. Anytime one of the servers fail, the load balancer stops forwarding requests to it. By intercepting the traffic, load balancers allow you to perform [rolling upgrades](#) one server at a time without any downtime.

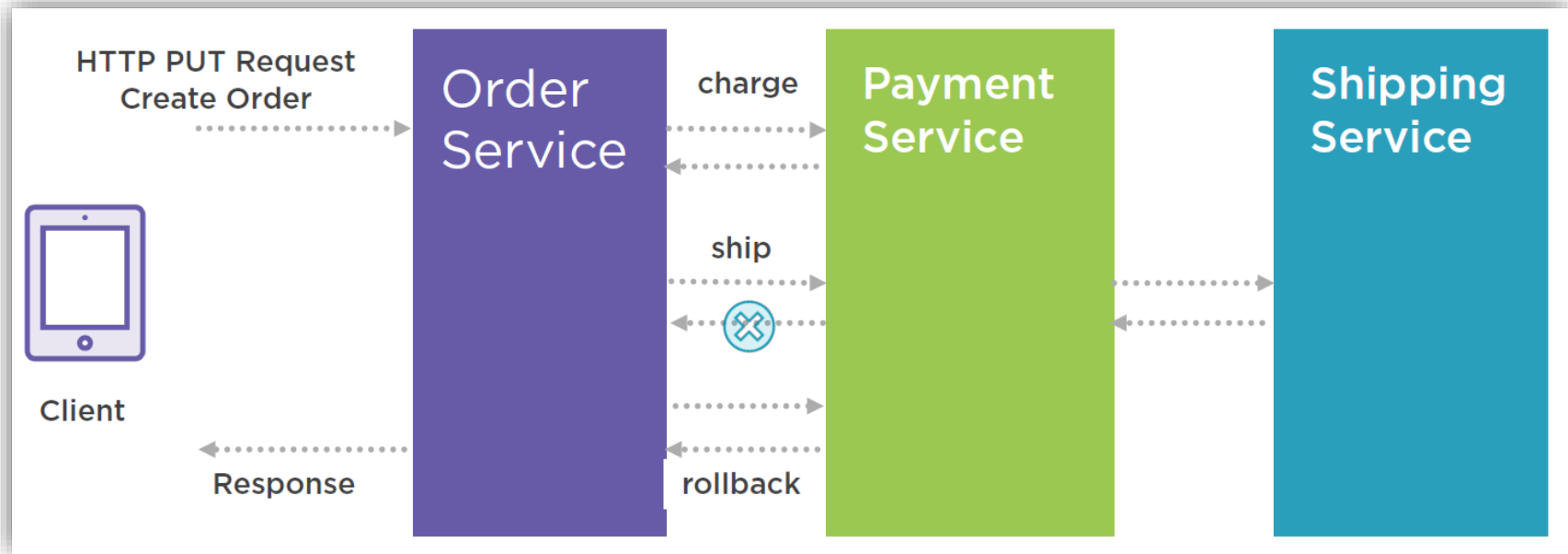
Implementing an operation consisting of a set of web service calls that should either fail or pass?

Let's look at some of the disadvantages of moving the state out from your servers. A distributed transaction is similar to a transaction in a database. It's an operation that changes the state of your application and consists of multiple web service calls across the network. The change is permanent only when all of the web service calls are successful. Otherwise, the transaction is aborted, and no change is applied.

Let's look at an example. On our Globomantics E-Commerce platform, when the buyer completes checkout, the client might invoke a PUT request on the order web service to create the order. The order service depends on the payment service and the shipping service. For the order to be successfully created, all of these requests have to be successful. You can't ship an item if the payment was unsuccessful. All these services share resources for the duration of this transaction to allow rollbacks in case of failures. A common way to implement distributed transactions is through the [two-phase commit protocol](#). But due to the blocking nature of the protocol, you end up sacrificing high availability for scalability.

[Saga pattern](#)

This tradeoff is not acceptable for many companies who instead opt for a [Saga Pattern](#) using **compensating transaction**. In a compensating transaction, you revert the changes of the operation if it fails. In our example, let's consider the scenario when the payment service call succeeds while the call to shipping service fails. In this case, the order service makes an additional call to the payment service to process the refund. None of the services have to wait for each other or hold any resource locks for the duration of the transaction.

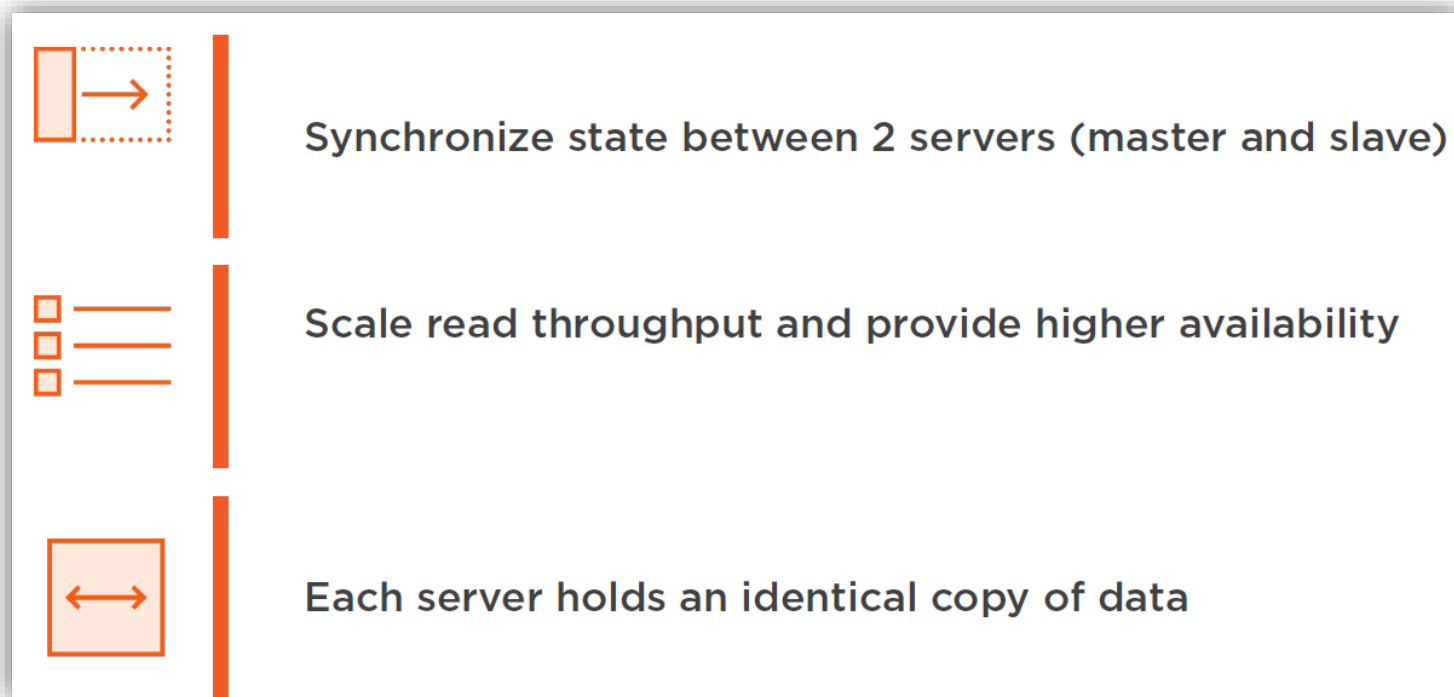


Article discussing distributed transactions (4 parts): [linkedin.com/pulse/microservices-distributed-transaction-management](https://www.linkedin.com/pulse/microservices-distributed-transaction-management)

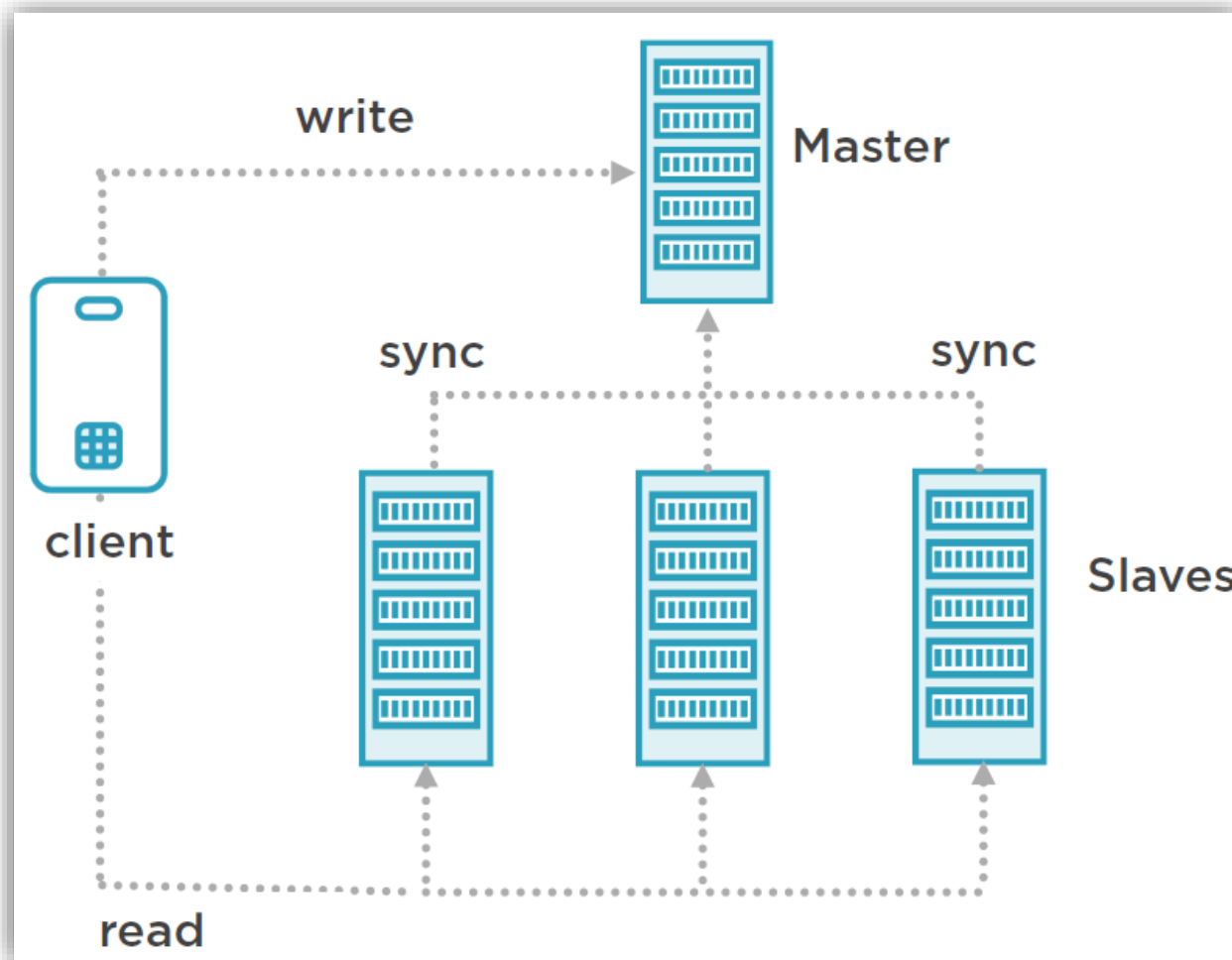
Data Layer

The data layer can often be the bottleneck in any distributed system. In order to scale the data layer, you need to understand the needs of your application. The data layer for a URL shortening service will look significantly different than the one for a social networking website. It also influences whether you offer a relational database, NoSQL database or a combination of both. Let's look at some of the techniques that will help you scale the data layer.

Replication



With [replication](#), you're constantly synchronizing the state between multiple servers, typically a primary and a secondary server. It allows your application to scale your read throughput and provide high availability. So if certain parts of the application are read intensive, they can independently be scaled by just adding secondary service. Each of these servers would hold a copy of the same data as the primary server. Example:



We have a client who's communicating with one primary and three secondary servers to store and retrieve data. The client talks to the primary server for all the write request, but anytime it wants to read data, it talks to the secondary server. This is also known as the primary-secondary replication where the secondary servers are synchronously connecting to the primary server in order to keep their copies in sync. If there's a sudden increase in the number of read requests, you can simply add more secondary servers to scale

horizontally. When designing a distributed system, it's always good to ask yourself, what can go wrong here? The primary server is also the single point of failure in our system. If it goes down, all your writes would start failing. This can be addressed with another configuration like the [primary-primary replication](#).

Replication Challenges

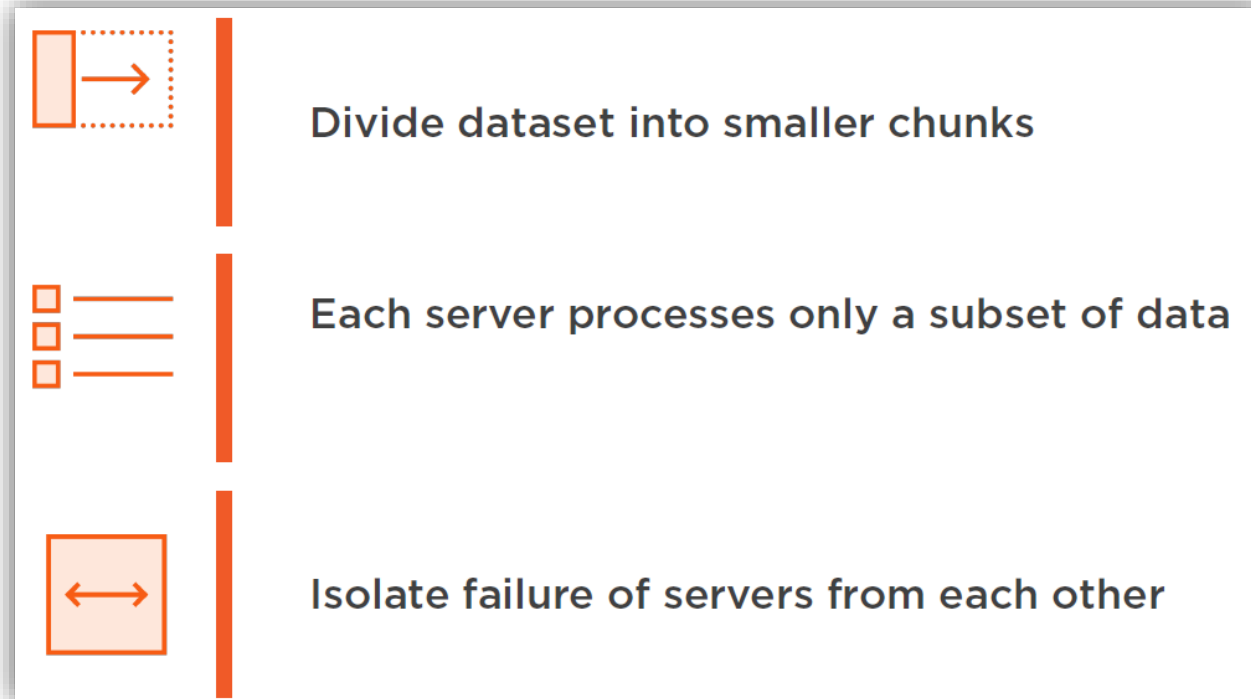
Introduces complexity

Replication lag

Only applicable for scaling reads

Replication adds a considerable amount of complexity around ensuring that data is consistent across all the replicated servers. It can also result in replication lag where servers are momentarily out of sync due to increased traffic or a slow network. Finally, replication only allows you to scale read requests. Let's look at how we can scale write requests, next.

Sharding / Partitioning

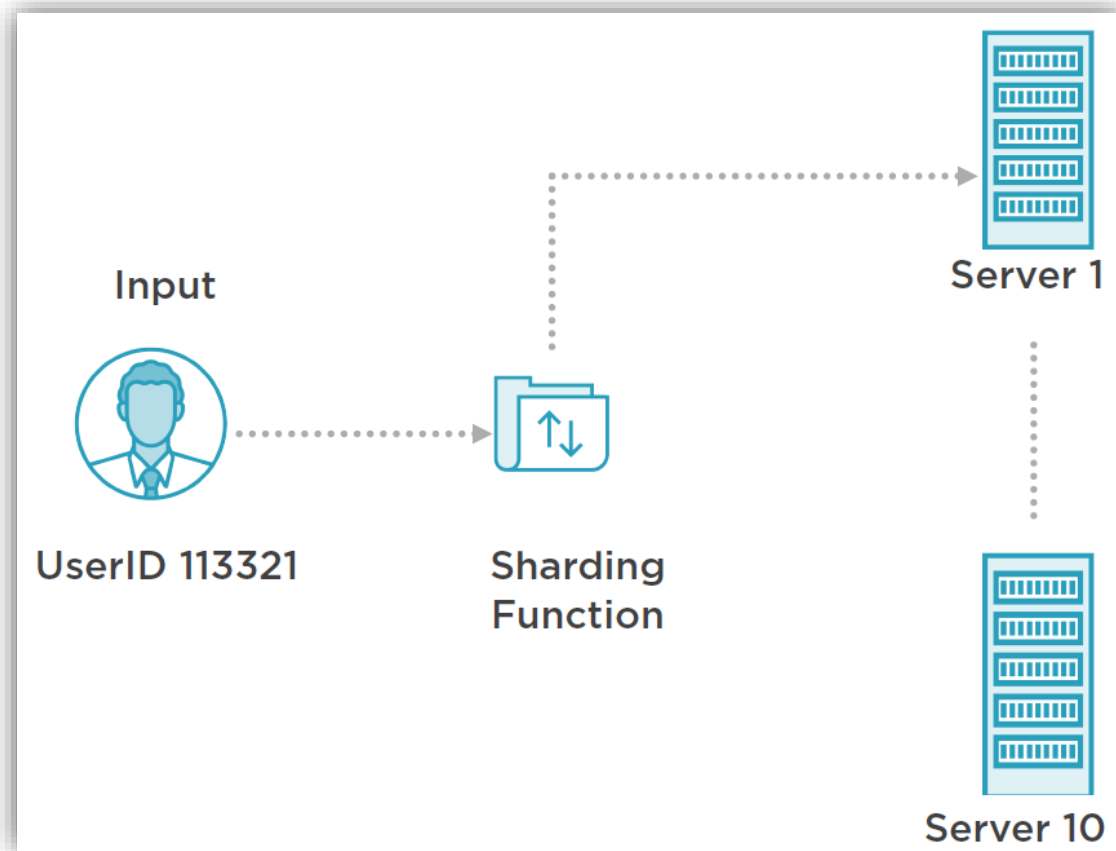


[Sharding](#), also known as data partitioning, involves dividing the dataset into smaller chunks and distributing it across multiple servers. Every server is only processing a subset of data at a time, allowing it to be independent of each other. This not only isolates the server from failures of other servers, but also eliminates the need for constant communication between them. So how exactly do we divide this dataset?

Sharding key / Partition key

We do this by identifying the sharding key. A sharding key determines how the dataset will be distributed among the servers in a cluster. These servers are also known as shards or partitions. In our e-commerce website, let's say we have 10 servers, and each buyer has a six-digit user ID. We can take the last digit of the user ID and mod it by the total number of servers and associate that

user with server number 1. So every time this buyer visits our website, that data can be read ordered into Server 1. This is a very basic implementation, and there are a number of other ways you can approach sharding.



Just like replication, there are a few caveats you should know when using sharding. Firstly, it can add a lot of complexity to your code, especially if you're implementing it on your own. In our previous example, as the data grows, we might add additional servers. What happens when you add or remove a server? Does it break our original user to server mapping? What if you need to retrieve

aggregated data for more than one user? This would require querying across multiple shards. This can severely degrade the performance of your server. If your application is hosted on Amazon AWS, then you can use Amazon relational database servers.

Sharding Challenges

Sharding function can be complex
Mappings can break
Querying data across shards

CAP Theorem

The [CAP theorem](#) states that it's impossible to build a distributed system that would simultaneously guarantee consistency, availability, and partition tolerance. A system is consistent when all servers see the same data at the same time. This is different from the concept of consistency, as defined in the asset properties of a relational database where consistency is primarily focused on the validity of data as it changes from one state to another. Availability guarantees that a server can process client requests even when other servers in the network are down. Partition tolerance ensures that the system can operate correctly even when servers cannot communicate with each other due to network failures. In a distributed system, the network is not reliable, and partitions can't be avoided.

Consistency

All servers see the same data

Availability

Server can process requests even when other servers are down

Partition Tolerance

System is functional even when servers cannot communicate

So for all practical purposes, you will be choosing between consistency and availability. This is when NoSQL databases come into the picture. A CP database delivers consistency and partition tolerance over availability, and AP database delivers availability and partition tolerance over consistency. MongoDB is a CP database. Cassandra is an AP database. MongoDB stores data as binary JSON documents. Cassandra stores data on a distributed network. In MongoDB, when there is a network failure between two servers, the server without consistent data shuts down until the network comes back up. In Cassandra, during a network failure, all servers remain available, but some servers may return all data. When the network connection is restored, these servers resync to get the latest data. This is also known as eventual consistency.

CP Database	AP Database
MongoDB	Cassandra
Stores data as BSON documents	Wide column database
Single Master-Slave configuration	Masterless configuration
Server shutdown during partition	Server remains available

A NoSQL database like Cassandra is ideal for a website like Twitter that needs to be highly available, but can tolerate eventual consistency. When a celebrity tweets, not every follower needs to see the tweet right away. If you're implementing the payment service for an e-commerce platform, then a relational database like MySQL would be preferable.

Extra materials

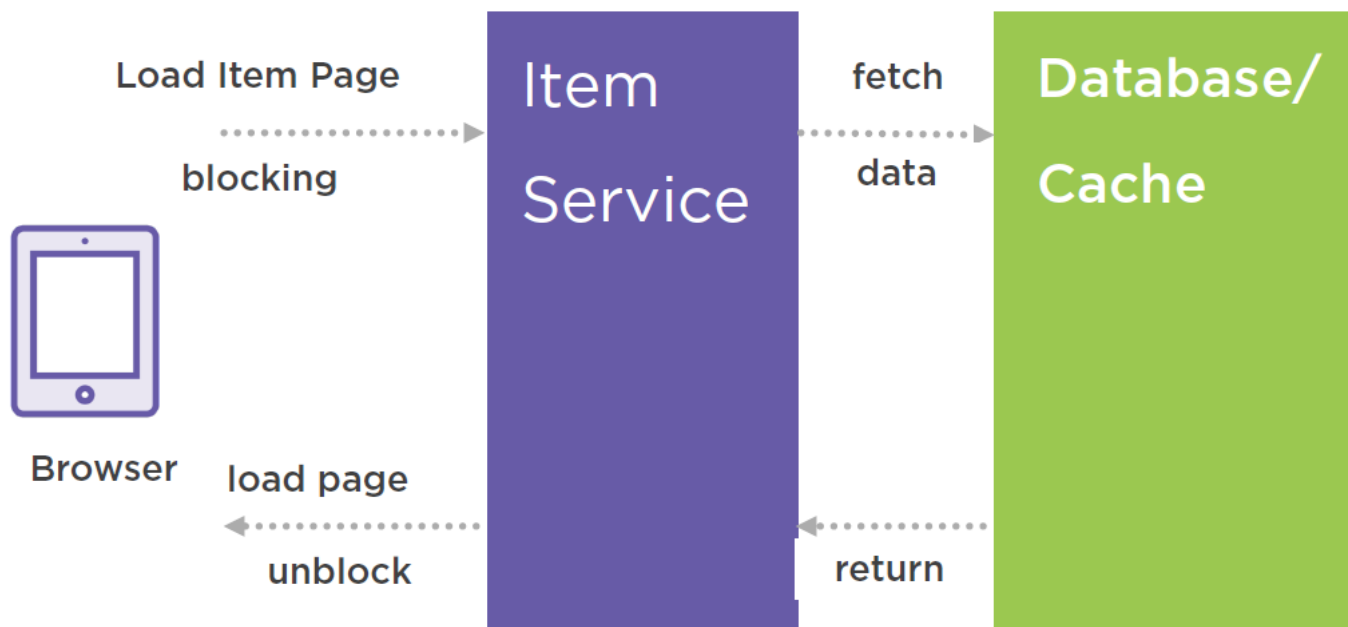
- CosmosDB lecture
- NoSQL lecture

Communication

Asynchronous communication is an important part of many distributed systems as not all operations in your application will be executed in a timely manner. This could be due to a slow network or the operation itself can be a long running one like a batch job that runs on a nightly schedule. In these situations, asynchronous processing allows us to build **loosely-coupled components**. Let's look at when communication should be synchronous or asynchronous.

Synchronous communication

With synchronous communication, a call is made to a remote server which **blocks until the operation completes**. For example **client - server communication over HTTP protocol (request - response) using a REST API or GraphQL API**. An example of synchronous communication is when a user loads the Item page on an ecommerce website. As soon as the request for data is sent to the item service, the browser blocks until the item service fetches the data from the database or a cache. Once the data is found and returned to the service, the page is rendered on the browser with the item details. Synchronous communication is simple and great for operations that have a low processing time.



Asynchronous communication

With asynchronous communication, the **caller does not wait for the operation to complete** before returning. There are three approaches to asynchronous communication:

Fire and Forget

Caller doesn't care if the operation completes or not

Callback

Register a callback to be notified when the operation completes

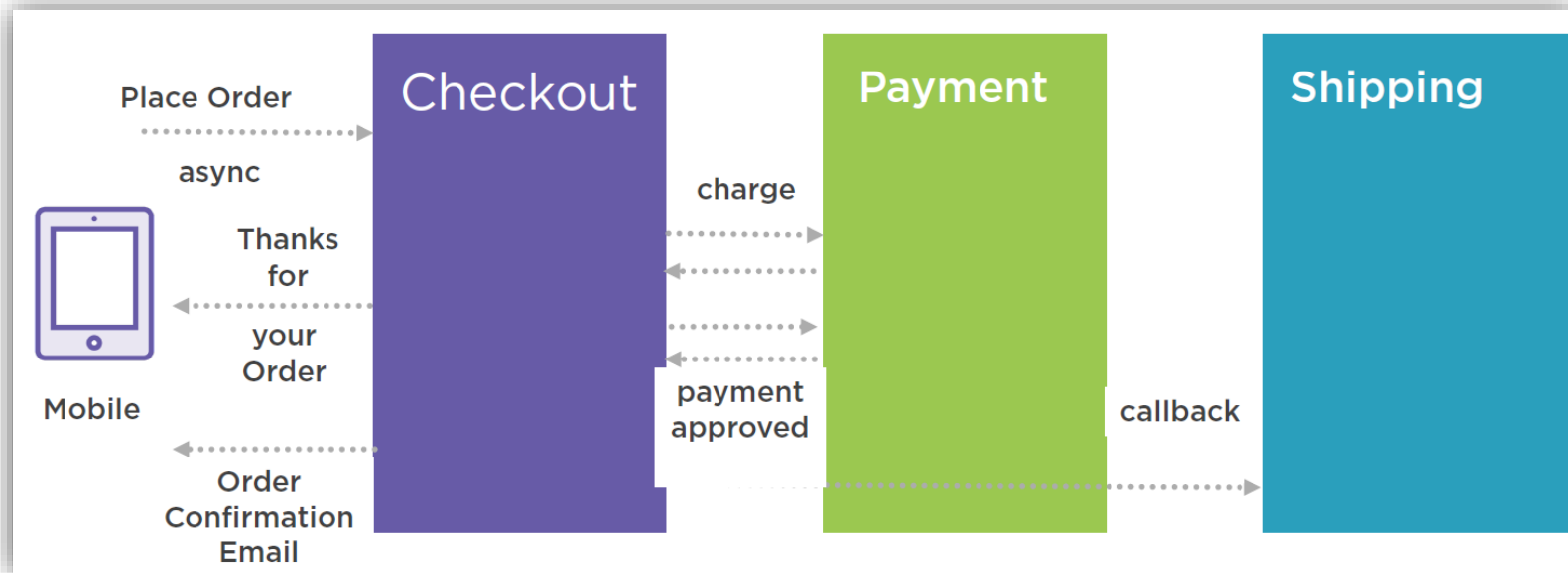
Event-based

Client emits events. Subscribers react independently

- [Fire and forget pattern](#) where the caller may not care when the operation takes place or even if it completes or not.
- **Callback pattern** where the caller might kick off an operation and register a callback asking the server to be notified when the operation has completed.
- **Event-based pattern** where instead of the client initiating request, asking for things to be done, it just emit events and expect other parties, also known as subscribers, to know what to do. And like the other two patterns, the business logic is more evenly spread out here. This makes the application highly decoupled. You can add or remove subscribers without affecting the client.

Example of asynchronous communication:

On a Globomantics ecommerce platform, when the buyer places an order, the client makes an asynchronous call to the checkout service and immediately returns with a message, thanking the buyer for their order. At this point, the order is not confirmed. Several operations are kicked off in the background, checking the inventory, calling the payment service to verify if a payment is valid, notifying the seller, and so on. If this call was synchronous, the client will be blocked until all of the steps are completed. Instead, we send the buyer an email with the order confirmation details once all the steps are successful. The checkout service may also register a callback to initiate shipping once the payment has been approved.



Message queues



Buffers and Distributes Asynchronous requests

Messages are platform independent (JSON, XML)

Producers and Consumers run as separate processes and hosted on separate servers

The [message queue](#) component buffers and distributes asynchronous requests. It allows you to achieve asynchronous processing, even if your application or programming runtime doesn't support it. It is common for the message to be represented in JSON or XML format. These messages are created by **producers** and processed by **consumers**. Producers and consumers run as completely separate processes and are hosted on separate servers. They are **completely independent of each other and interact through the message queue identified by an endpoint**. This allows them to be scaled independently.

Producers initiate the asynchronous calls and are the source of data in a queue. They are part of the client code. Their only responsibility is to create a message, serialize it before pushing it to the message queue. It can be implemented in any language and doesn't care if there is a consumer available to consume its messages or not. **The only contract between the producer and consumer is the message format**. So defining the message in a platform agnostic format like JSON or XML is usually the very first step.

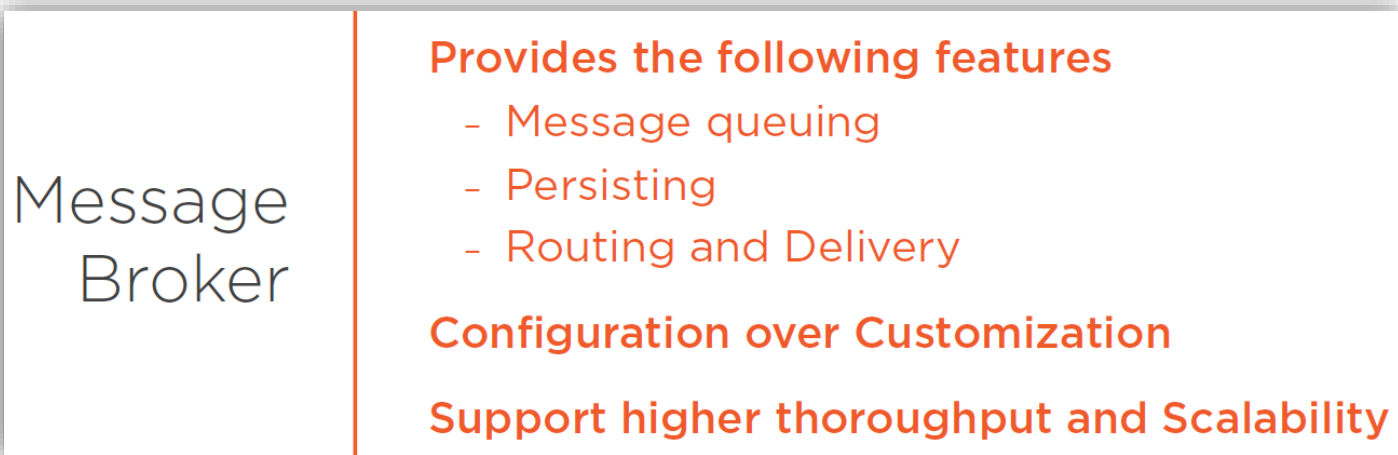
Producers	Initiate asynchronous calls Create a message and push it to the Queue Message format serves as the contract
-----------	--

Example of how a message format can be defined in XML:

```
<?xml version="1.0">
<message>
  <invoice date="01-15-2020" id="321332">
    <item>1234432321</item>
    <title>A Brief History of Nearly Everything – Bill Bryson</title>
    <USPrice>12.99</USPrice>
```

You could have a producer in client code written in JavaScript creating this message with the invoice details anytime an order is created. On the other side, you could have a consumer in the form of a batch job written in Java processing this message and storing the invoice details in a data warehouse for analytics.

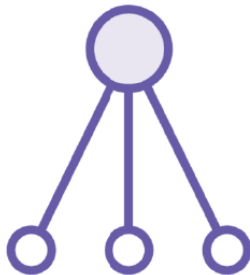
The message queue is usually implemented as a specialized application known as a [message broker](#). It provides several features out of the box, like the buffering and queuing of messages, persisting them just like a database, routing these messages between producers and consumers. Message brokers are meant to be configured rather than customized. They are a lot simpler than a database engine, which is why they are able to support higher throughput and scalability.



Consumers are like producers to the actual asynchronous processing. This makes them a lot more complex than the producers which only creates the message. They are implemented in the application code where our business logic resides and not the client code. In a pull model implementation, the consumer periodically connects to the queue to check for messages, and if available, starts consuming them. This model is used by consumers implemented in scripting languages like PHP or when the network is unstable. In a push model implementation, the consumer opens and maintains a connection with the message broker. This model is used by consumers implemented in languages like Node.js or Java that have a persistent application container. Messages are pushed to the consumer by the message broker as soon as they arrive.

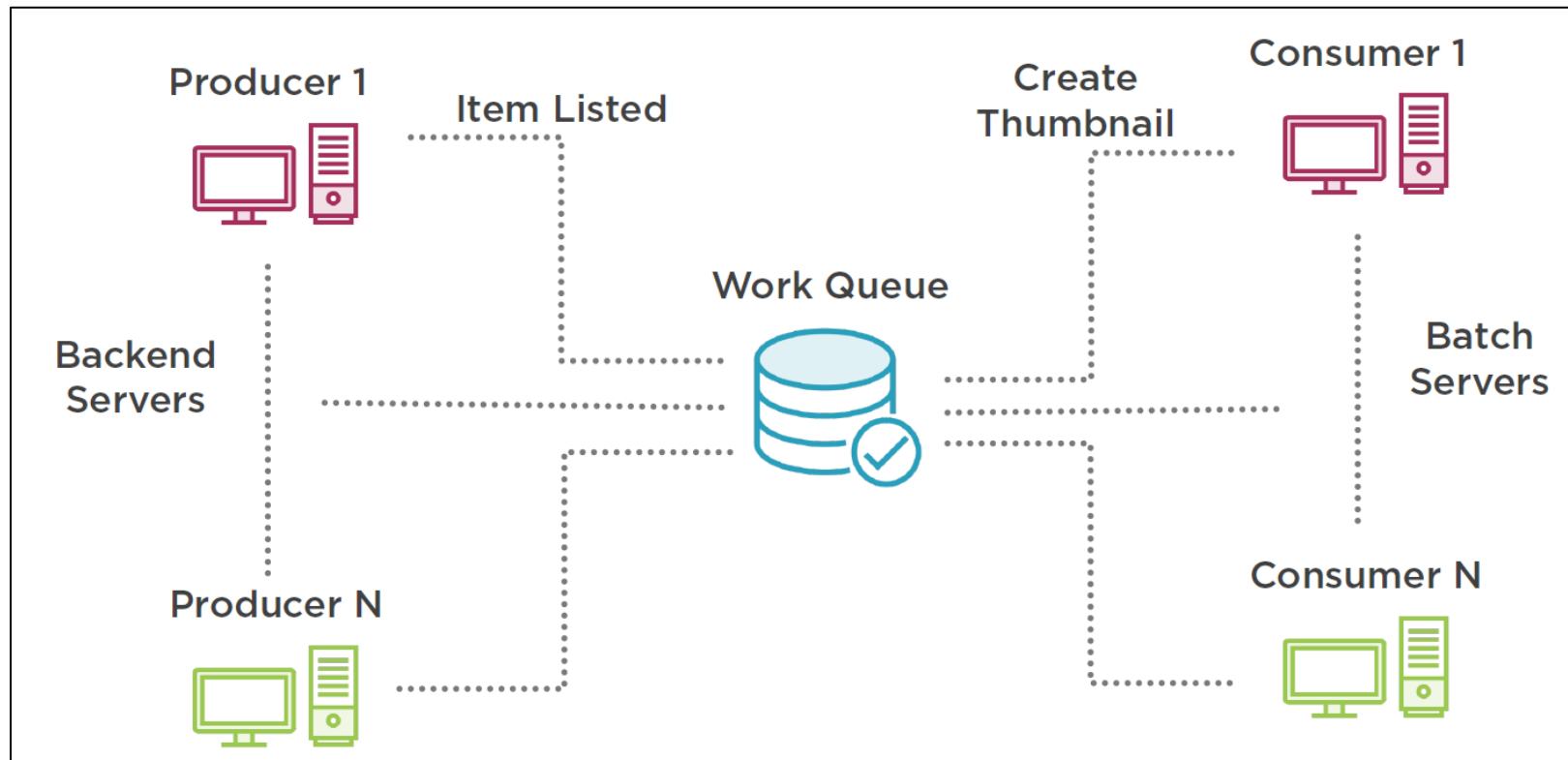
Consumers	Process requests asynchronously Implemented in the application code - Pull model - Push model
-----------	--

In addition to the [push and pull models](#), message brokers provide various routing methods to give consumers more control and how they can listen to messages. Let's take a look at one of these methods next. Here is the direct worker queue routing method. There is a single queue identified by its name. One or more producers publish messages to the work queue. Each message is then routed to only one consumer. Both the producers and consumers can be scaled horizontally. The direct worker queue method is ideal for tasks that take a long time to process. For example, a website like Netflix may want to transcode the entire video library and make it available at different resolutions for viewers with different devices and bandwidth. Similarly, classifying data to find relationships between them can also take a lot of time. Another use case that is relevant to our Globomantics ecommerce platform is generating the thumbnails for every item listed on the site. This thumbnail can then be shown in research results instead of the actual high resolution image. This can save us a lot of bandwidth.

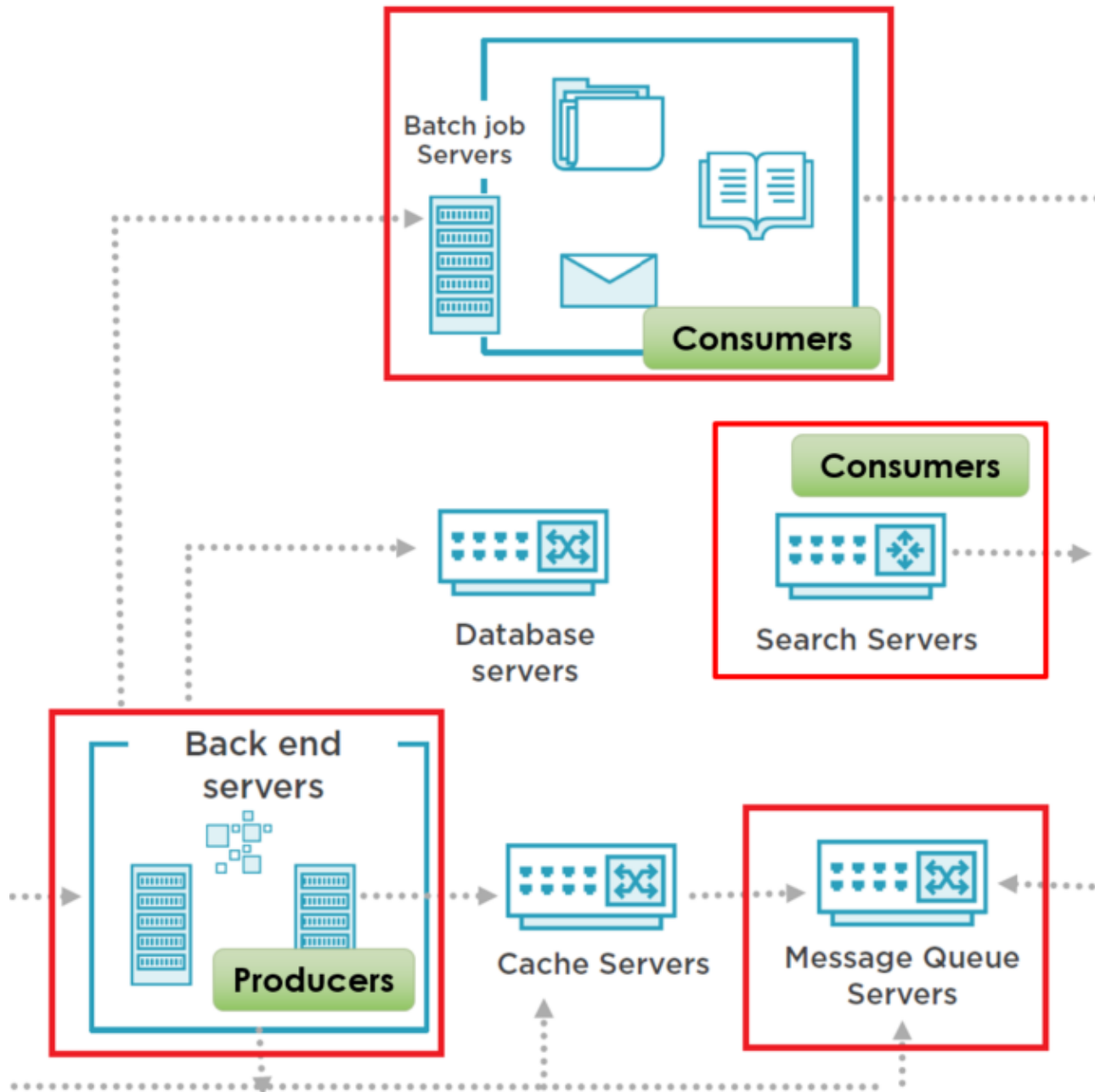


- Single Work Queue identified by name**
- Multiple Producers and Consumers**
- Each Message routed to one Consumer**
- Ideal for long running tasks**
 - Video Transcoding
 - Data classification
 - Image resizing

Let's see how this may look like. Our backend service can also act as producers that create and push messages to a work queue every time an item is listed on our platform. Each message is then routed to a consumer for performing the resize operation. These consumers are part of the batch server section of the overall system diagram as shown in the next slide.



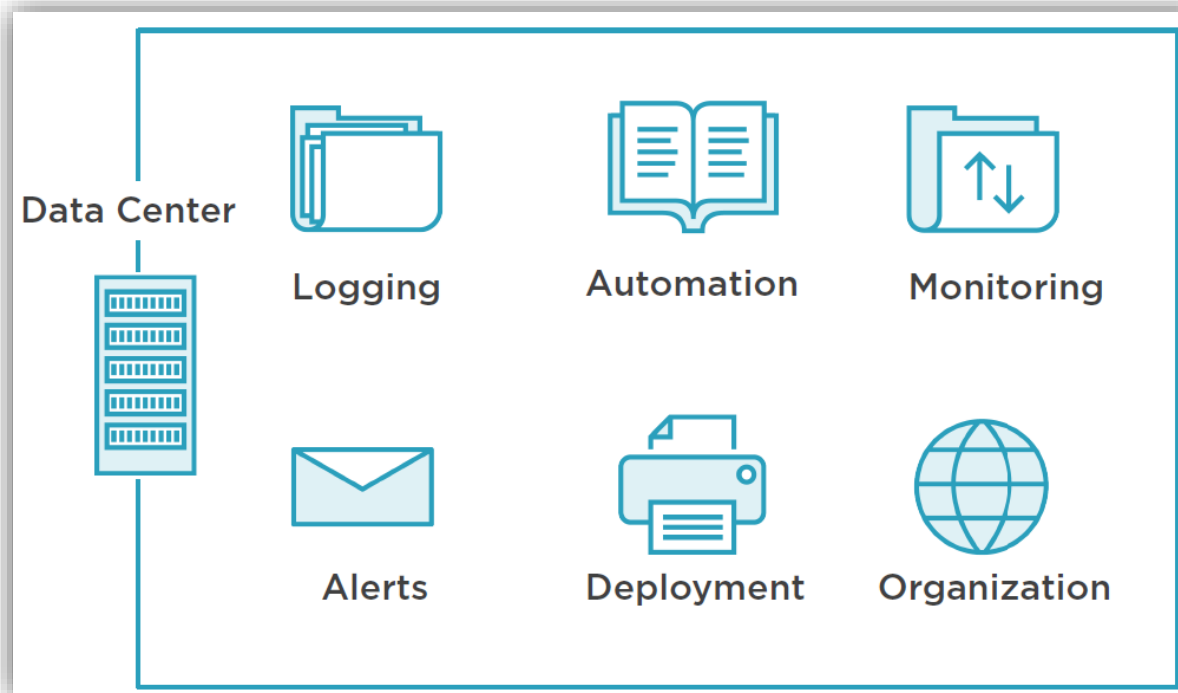
As you can see on the next figure, both the backend service and the batch job servers communicate with the message queue servers. They are completely independent and operate asynchronously. Additionally, you can have several different consumers performing a range of tasks. For example, the search servers here are also consuming a different set of messages from the message queue. These servers could be an [Elasticsearch](#) cluster indexing user behavior events like page clicks and impressions. You can then use a tool like [Kibana](#) to search for these events and generate graphs tracking how users spend their time on your site.



Distributed Systems in the Real World

It's equally important to understand what it takes to maintain Distruted systems in the real world. In traditional systems, you are limited to development, testing, and deployment of a handful of servers.

Adopting the distributed model not only increases the complexity of the application and infrastructure, but also presents very unique challenges on how to keep them up and running. The breadth and impact of a distributed architecture spans beyond just the database and servers it's made up of. Here are a number of moving parts working together to realize the function of your application. These include logging and monitoring, adopting a culture of automation, setting up alerts to know when things go wrong, independently deployable services, and modeling the teams within your organization around the business and technical domains. When all these are integrated, you have a system that is resilient to failures, easier to scale, and highly efficient.



Scaling Teams and Organization

Scalability goes beyond just scaling your servers and storage. As your application grows, so does the size of your organization and teams. Hiring more engineers wouldn't necessarily get more done. On the contrary, it'll expose and magnify any inefficiencies existing in your organization. The same applies to operations. This is especially true if your application is hosted on a private or hybrid cloud. It can become difficult to monitor, deploy, and maintain hundreds of servers efficiently.

Scaling Teams

Avoiding inefficiencies when expanding workforce

Scaling Operations

Horizontal Scaling resulting in hundreds of additional Servers

Distributed systems also necessitates more interaction and collaboration among your team members. Delegating tasks and sharing responsibilities becomes an extremely important part of your day-to-day job. This is where practices like **pair programming** and **code reviews** can actively promote knowledge sharing within the team. You'll also be required to have frequent spontaneous discussions around design and architecture of your application to understand any bottlenecks that may prevent it from scaling. When there are production issues, it's important to perform [root cause analysis](#) and implement procedures to avoid similar issues in the future. Finally, new frameworks, tools, and technologies come out every few months, so it's vital that you keep up with the changes and adopt solutions that may assist your team with their scalability goals.

Pair Programming

Better Collaboration.
Developing a shared
knowledgebase

Design Discussions

Identifying bottlenecks.
Performing RCA. Keeping up
with change in Technology

Automating Testing and Deployment

Distributed systems can add an incredible amount of complexity due to the sheer number of moving parts in your application. Adopting a culture of automation helps us address this complexity by encouraging us to work on the tooling and infrastructure needed.

Automated Testing

Unit, Integration and
Performance Tests

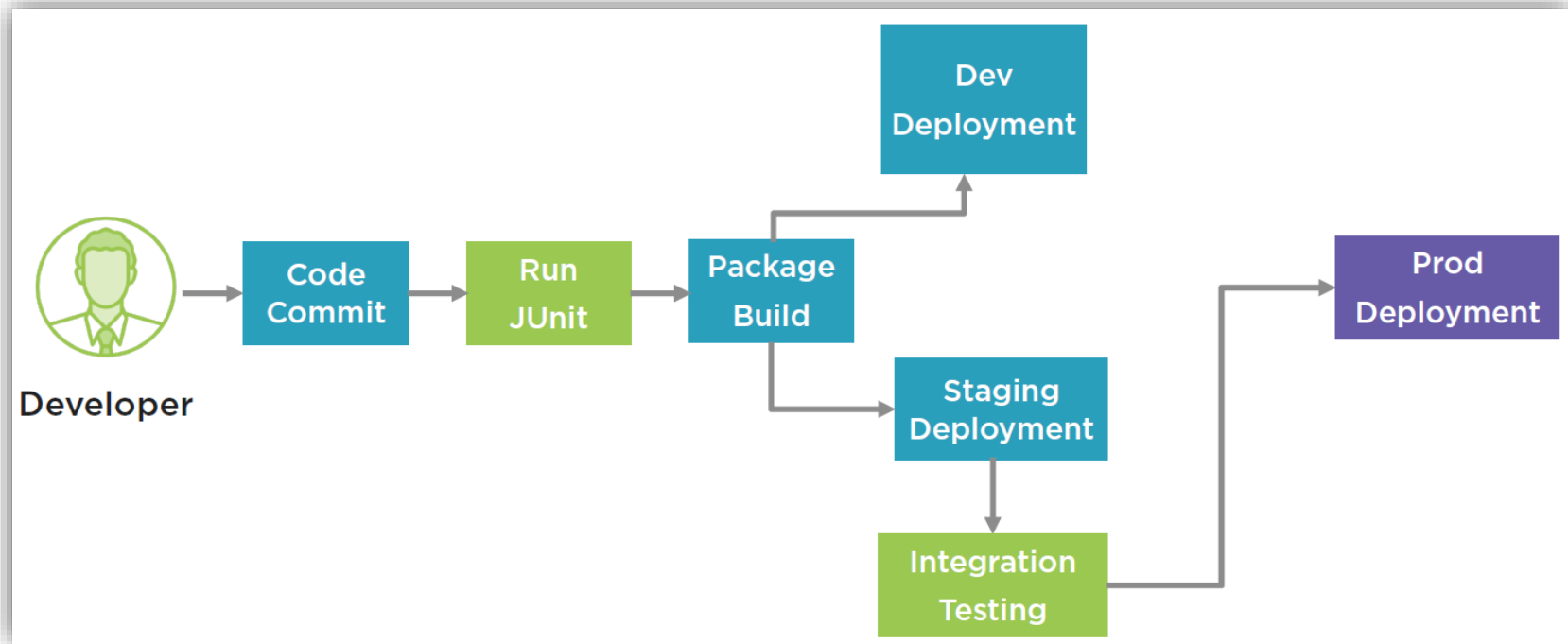
Continuous Delivery

Automate Build and
Deployment

Use Server Definitions

Re-create environments
on demand

- Automated testing is extremely important and gives us the confidence that what we are building and releasing works as expected. Without it, we could end up spending more time constantly fixing issues instead of developing new application features for our customers.
- Continuous delivery allows us to automate the entire build, test, and deployment pipeline, allowing us to release to production at any time. This helps us get fast, automated feedback on the quality of production releases. Ideally, you should be able to confidently deploy the current development build into production at a moment's notice.
- The use of server definitions gives us the ability to specify and deploy to multiple environments. For instance, you may first develop and deploy a build to a feature pool and test only specific features, Then, you can deploy the same build to a staging environment that is very similar to the production environment. All of this should be possible by essentially using a uniform deployment method.



Here's an example of a continuous delivery pipeline. It begins with the developer committing code to an integration branch in the repository. This should trigger the execution of the test suite followed by the creation of the build. This build would then be deployed to development and staging environment. Next, the integration test suite would be run against the staging environment. Finally, if each of these steps in the pipeline are successful, the build would then be deployed to the production environment. The entire pipeline should ideally be automated, requiring no intervention.

Setting up Monitoring and Alerting

The only way to ensure that your system is consistently up and running is through automated monitoring and alerting. This enables us to identify issues and take corrective action in a timely manner and keep the business impact to a minimum.



Firstly, we want to limit the time it takes between the first occurrence of the problem and its identification. We don't want to rely on our customers to report the problem, but be proactive in putting automated systems in place that allow us to discover the issue in a matter of minutes.

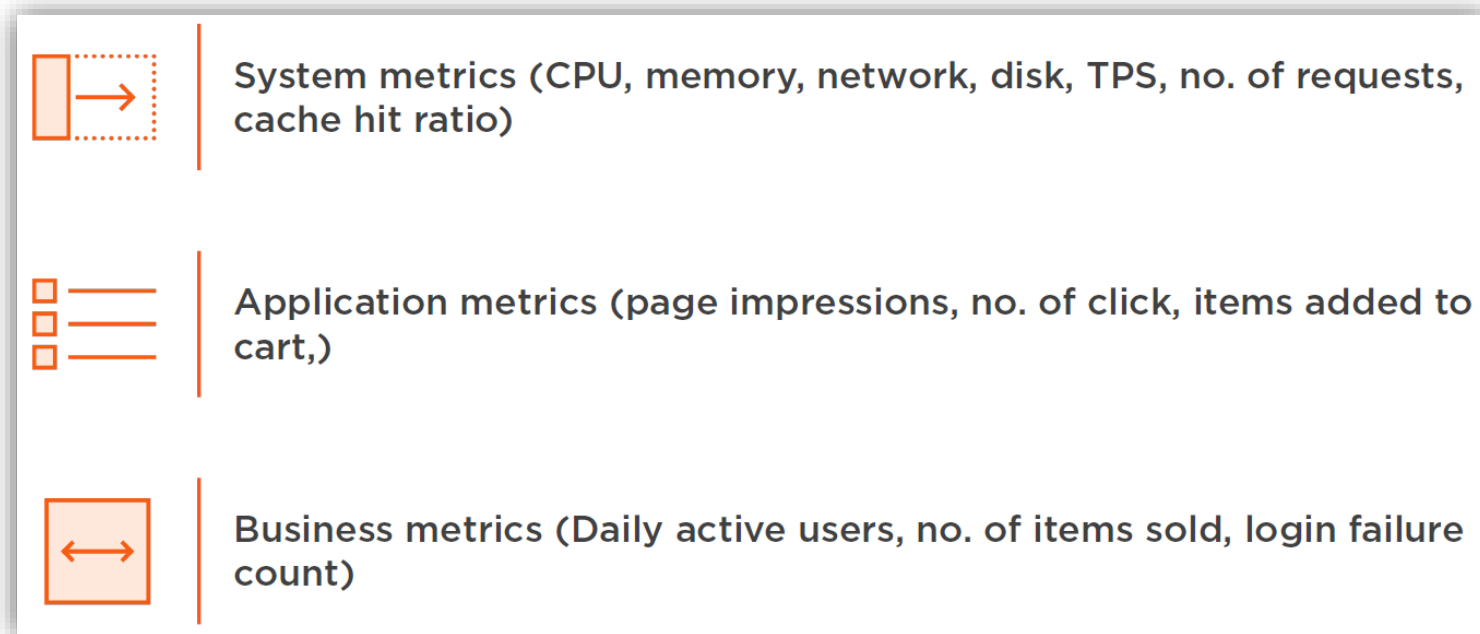
We need to identify the people with the technical know-how to be alerted and start looking into the problem. A lot of companies have an on-call rotation program to precisely address this issue. With services like [PagerDuty](#), this can be easily implemented.

The next step involves setting up clear and well-established procedures to allow the on-call engineer to resolve the issue. This may involve having a direct line of access to the site reliability engineering team or a technical duty officer who has the necessary access privileges to run database queries in production or nuke server instances.

Finally, after the issue has been investigated by looking through server and application logs, a fix needs to be made and rolled out to production as quickly and safely as possible. At this point, it's important to consider the impact of the change so that the fix itself doesn't introduce additional issues. This is where a continuous delivery pipeline can be invaluable. All of this assumes you have the right monitoring, alerting, and logging systems in place collecting the relevant logs and metrics. Let's explore metrics collection next.

Collecting and Viewing Metrics

We are primarily concerned with collecting three types of metrics.

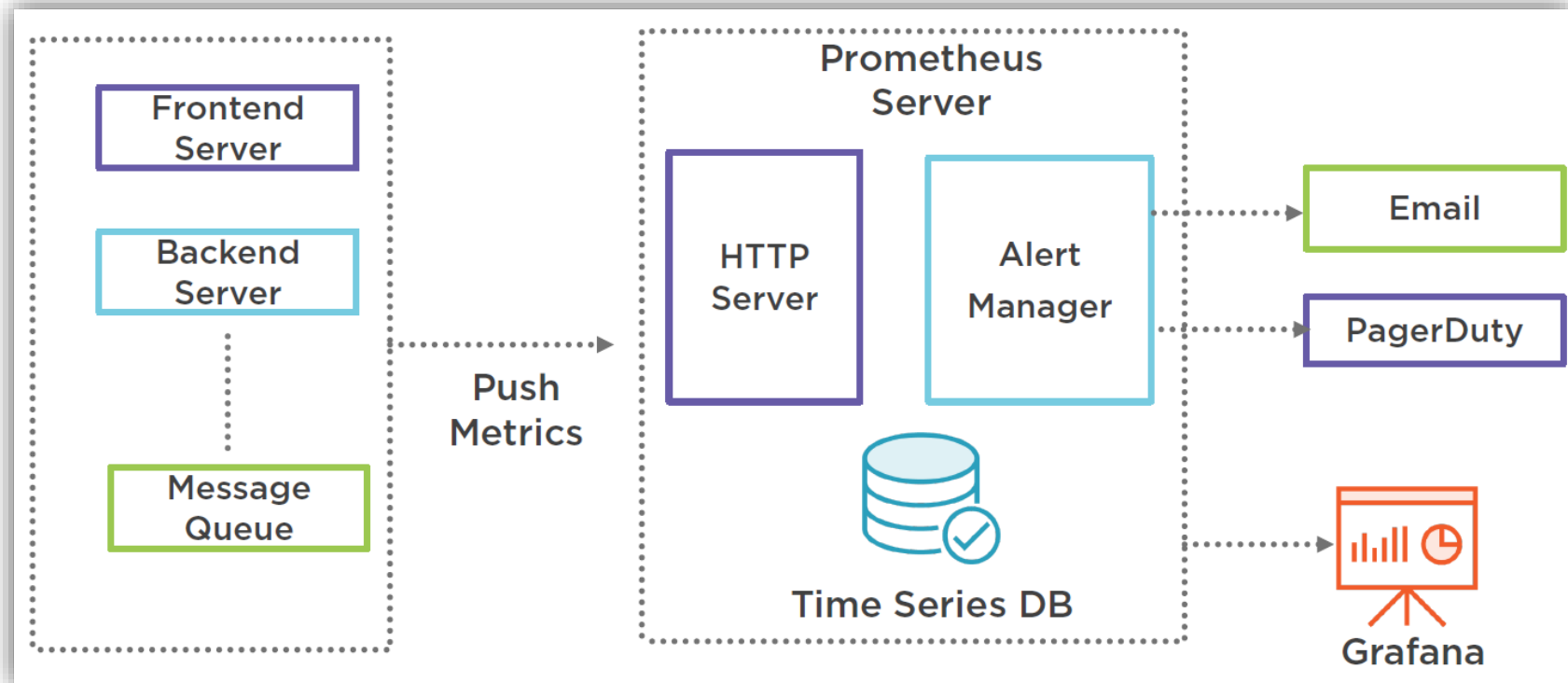


System metrics include data related to the hardware, network, operating system, and the server instances. These include information like the load on CPU, memory usage, network connections, and Disk I/O. They also include the metrics relative to the server instances like the database transactions per second, number of web requests per second, and the cache hit ratio.

Next, we have the metrics relative to the **application** itself. These could be events associated with user behavior like page impressions, number of clicks on a module, number of items added to shopping cart, and so on. This can really help your developers understand how the users are interacting with the application.

Then we have the metric that directly impacts the **business**. These include daily active users on your website, total items sold, and user login failure count. This allows senior management to direct their effort where it's most needed based on actual data.

Real world example:



This is one of the ways a monitoring and alerting solution could be configured in production. On the left, we have all the servers deployed with monitoring agents pushing various metrics to the centralized [Prometheus](#) monitoring server. This server comes with its own [time series database](#) that serves as the storage for all the metrics that have been collected. It also has an Alert Manager that triggers alerts based on user defined rules. These alerts can take the form of an email or push notification on a mobile app like PagerDuty. Finally, we can use a tool like [Grafana](#) to create graphs and dashboards on all the metrics collected by Prometheus. This is especially useful to display trends.

Frameworks and libraries for developing distributed systems

Java

- Spring Boot
- Quarkus
- Akka

C#

- ASP .NET
- ASP .NET Core

Python

- FastAPI
- Minos

JavaScript

- <https://geekflare.com/javascript-frameworks-for-api/>
- <https://www.npmjs.com/package/moleculer>
- <https://moleculer.services/docs/0.12/examples.html>
- <https://github.com/moleculerjs/project-demo>

Go

- gokit

Links

JWT vs Session: <https://www.youtube.com/watch?v=UBUNrFtufWo>

DNS: <https://www.youtube.com/watch?v=UVR9lhUGAyU>

Architecting applications on Azure: <https://learn.microsoft.com/en-us/azure/architecture/>

Load Balancer: <https://azure.microsoft.com/en-us/products/load-balancer>

Data partitioning: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/data-partitioning>

REST: https://en.wikipedia.org/wiki/Representational_state_transfer

Rolling updates with Kubernetes: <https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/>

Saga Pattern: <https://learn.microsoft.com/en-us/azure/architecture/reference-architectures/saga/saga>

CAP Theorem: https://en.wikipedia.org/wiki/CAP_theorem

Message Brokers:

- RabbitMQ: <https://www.rabbitmq.com/>
- Kafka: <https://kafka.apache.org/>

Redis: <https://redis.com/>

Distributed transactions in microservices (linkedin article - 4 parts): [linkedin.com/pulse/microservices](https://www.linkedin.com/pulse/microservices)

Database geo-replication: <https://learn.microsoft.com/en-us/azure/azure-sql/database/active-geo-replication>

Database sharding: <https://www.digitalocean.com/community/tutorials/understanding-database-sharding>

Cloud Design Patterns - Microsoft: <https://learn.microsoft.com/en-us/azure/architecture/patterns/>

System Metrics, Monitoring, Alerting: [digitalocean.com/community/tutorials/an-introduction-to-metrics-monitoring-and-alerting](https://www.digitalocean.com/community/tutorials/an-introduction-to-metrics-monitoring-and-alerting)

Prometheus: <https://prometheus.io/>